

Suffix Arrays

Suffix array

$T\$ = \text{abaaba\$}$ ← As with suffix tree,
 T is part of index

$SA(T) =$
(SA = "Suffix Array")

6	\$
5	a \$
2	a a b a \$
3	a b a \$
0	a b a a b a \$
4	b a \$
1	b a a b a \$

$m + 1$
integers

Suffix array of T is an array of integers in $[0, m]$ specifying the lexicographic order of $T\$$'s suffixes

Another Example Suffix Array

$s = \text{cattcat\$}$

- Idea: lexicographically sort all the suffixes.
- Store the starting indices of the suffixes in an array.

0	cattcat\$
1	attcat\$
2	ttcat\$
3	tcat\$
4	cat\$
5	at\$
6	t\$
7	\$

index of suffix

suffix of s

sort the suffixes
alphabetically



the indices just
“come along for
the ride”

7	\$
5	at\$
1	attcat\$
4	cat\$
0	cattcat\$
6	t\$
3	tcat\$
2	ttcat\$

Another Example Suffix Array

$s = \text{cattcat\$}$

- Idea: lexicographically sort all the suffixes.
- Store the starting indices of the suffixes in an array.

0	cattcat\$
1	attcat\$
2	ttcat\$
3	tcat\$
4	cat\$
5	at\$
6	t\$
7	\$

index of suffix

suffix of s

sort the suffixes
alphabetically



the indices just
“come along for
the ride”

7
5
1
4
0
6
3
2

Table I. Performance Summary of the Construction Algorithms

Algorithm	Worst Case	Time	Memory
Prefix-Doubling			
MM [Manber and Myers 1993]	$O(n \log n)$	30	$8n$
LS [Larsson and Sadakane 1999]	$O(n \log n)$	3	$8n$
Recursive			
KA [Ko and Aluru 2003]	$O(n)$	2.5	$7-10n$
KS [Kärkkäinen and Sanders 2003]	$O(n)$	4.7	$10-13n$
KSPP [Kim et al. 2003]	$O(n)$	—	—
HSS [Hon et al. 2003]	$O(n)$	—	—
KJP [Kim et al. 2004]	$O(n \log \log n)$	3.5	$13-16n$
N [Na 2005]	$O(n)$	—	—
Induced Copying			
IT [Itoh and Tanaka 1999]	$O(n^2 \log n)$	6.5	$5n$
S [Seward 2000]	$O(n^2 \log n)$	3.5	$5n$
BK [Burkhardt and Kärkkäinen 2003]	$O(n \log n)$	3.5	$5-6n$
MF [Manzini and Ferragina 2004]	$O(n^2 \log n)$	1.7	$5n$
SS [Schürmann and Stoye 2005]	$O(n^2)$	1.8	$9-10n$
BB [Baron and Bresler 2005]	$O(n \sqrt{\log n})$	2.1	$18n$
M [Maniscalco and Puglisi 2007]	$O(n^2 \log n)$	1.3	$5-6n$
MP [Maniscalco and Puglisi 2006]	$O(n^2 \log n)$	1	$5-6n$
Hybrid			
IT+KA	$O(n^2 \log n)$	4.8	$5n$
BK+IT+KA	$O(n \log n)$	2.3	$5-6n$
BK+S	$O(n \log n)$	2.8	$5-6n$
Suffix Tree			
K [Kurtz 1999]	$O(n \log \sigma)$	6.3	$13-15n$

Time is relative to MP, the fastest in our experiments. Memory is given in bytes including space required for the suffix array and input string and is the average space required in our experiments. Algorithms HSS and N are included, even though to our knowledge they have not been implemented. The time for algorithm MM is estimated from experiments in Larsson and Sadakane [1999].

Suffix array: querying

Is P a substring of T ?

1. For P to be a substring, it must be a prefix of ≥ 1 of T 's suffixes
2. Suffixes sharing a prefix are consecutive in the suffix array

6	\$
5	a \$
2	a a b a \$
3	a b a \$
0	a b a a b a \$
4	b a \$
1	b a a b a \$

Suffix array: querying

Is P a substring of T ?

1. For P to be a substring, it must be a prefix of ≥ 1 of T 's suffixes
2. Suffixes sharing a prefix are consecutive in the suffix array

Use binary search

6	\$
5	a \$
2	a a b a \$
3	a b a \$
0	a b a a b a \$
4	b a \$
1	b a a b a \$

Suffix array: binary search

We can do the same thing for a sorted list of suffixes:

```
from bisect import bisect_left, bisect_right

t = 'abaaba$'
suffixes = sorted([t[i:] for i in xrange(len(t))])

st, en = bisect_left(suffixes, 'aba'),
bisect_left(suffixes, 'abb')

print(st, en) # output: (3, 5)
```

6	\$
5	a \$
2	a a b a \$
3	a b a \$
0	a b a a b a \$
4	b a \$
1	b a a b a \$

First place to insert the *smallest string* > query ('aba')

Python example: <http://nbviewer.ipython.org/6753277>

Suffix array: querying

Is P a substring of T ?

Do binary search, check whether P is a prefix of the suffix there

How many times does P occur in T ?

Two binary searches yield the range of suffixes with P as prefix; size of range equals # times P occurs in T

Worst-case time bound?

$O(\log_2 m)$ bisections, $O(n)$ comparisons per bisection, so $O(n \log m)$

6	\$
5	a \$
2	a a b a \$
3	a b a \$
0	a b a a b a \$
4	b a \$
1	b a a b a \$

Suffix array: querying

Consider further: binary search for suffixes with P as a prefix

Assume there's no \$ in P . So P can't be equal to a suffix.

Initialize $l = 0$, $c = \text{floor}(m/2)$ and $r = m$ (just past last elt of SA)



Notation: We'll use $SA[l]$ to refer to the suffix corresponding to suffix-array element l . We could write $\tau[SA[l]:]$, but that's too verbose.

Throughout the search, invariant is maintained:

$$SA[l] < P < SA[r]$$

Suffix array: querying

Throughout search, invariant is maintained:

$$\text{SA}[l] < P < \text{SA}[r]$$

What do we do at each iteration?

Let $c = \text{floor}((r + l) / 2)$

If $P < \text{SA}[c]$, either stop or let $r = c$ and iterate

If $P > \text{SA}[c]$, either stop or let $l = c$ and iterate

When to stop?

$P < \text{SA}[c]$ and $c = l + 1$ - answer is c

$P > \text{SA}[c]$ and $c = r - 1$ - answer is c

Longest Common Prefix

The longest common prefix of two strings s, t is simply the length of the prefix they share prior to the first difference (or the termination of either string).

S	ACTTACAGACCGGAGAC
T	ACTTACAGACGGAGCTAGC

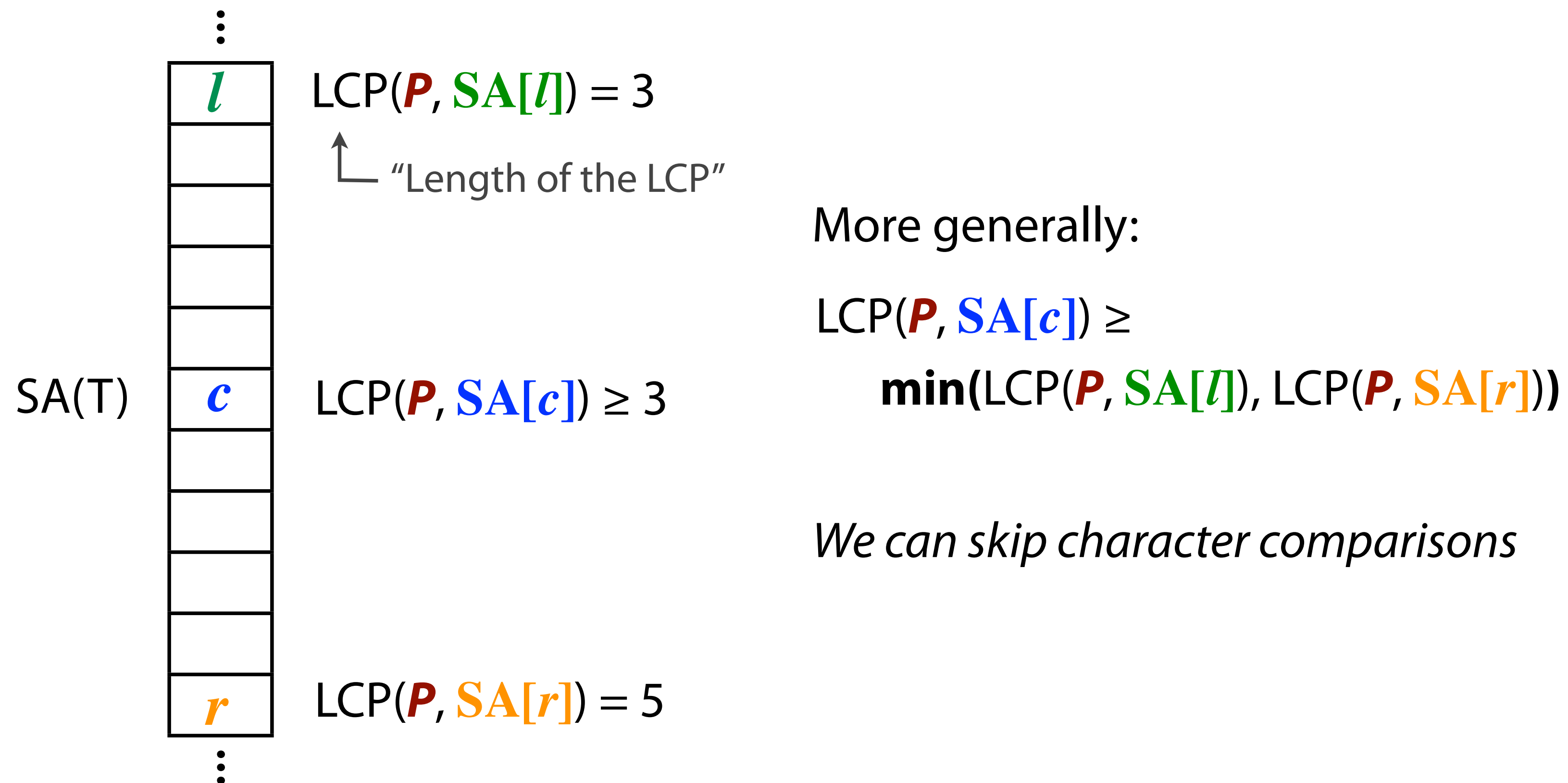
$\text{LCP}(S, T) = \text{ACTTACAGACGAC}$

$|\text{LCP}(S, T)| = 13$

Below, to avoid extra notation, we will use $\text{LCP}(S, T)$ as shorthand for $|\text{LCP}(S, T)|$

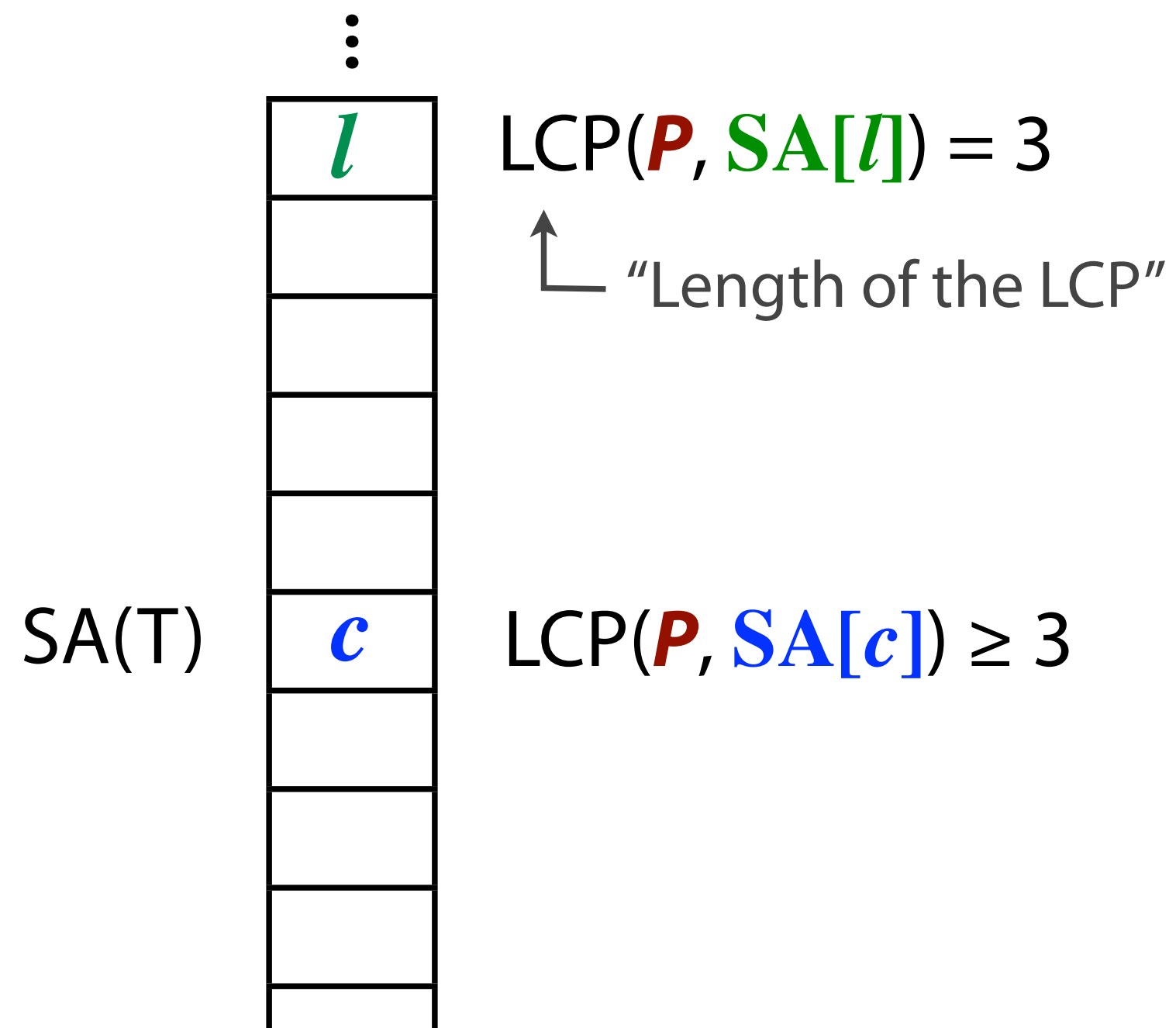
Suffix array: querying

Say we're comparing P to $SA[c]$ and we've already compared P to $SA[l]$ and $SA[r]$ in previous iterations.



Suffix array: querying

Say we're comparing P to $SA[c]$ and we've already compared P to $SA[l]$ and $SA[r]$ in previous iterations.



More generally:

$$LCP(P, SA[c]) \geq \min(LCP(P, SA[l]), LCP(P, SA[r]))$$

We can skip character comparisons

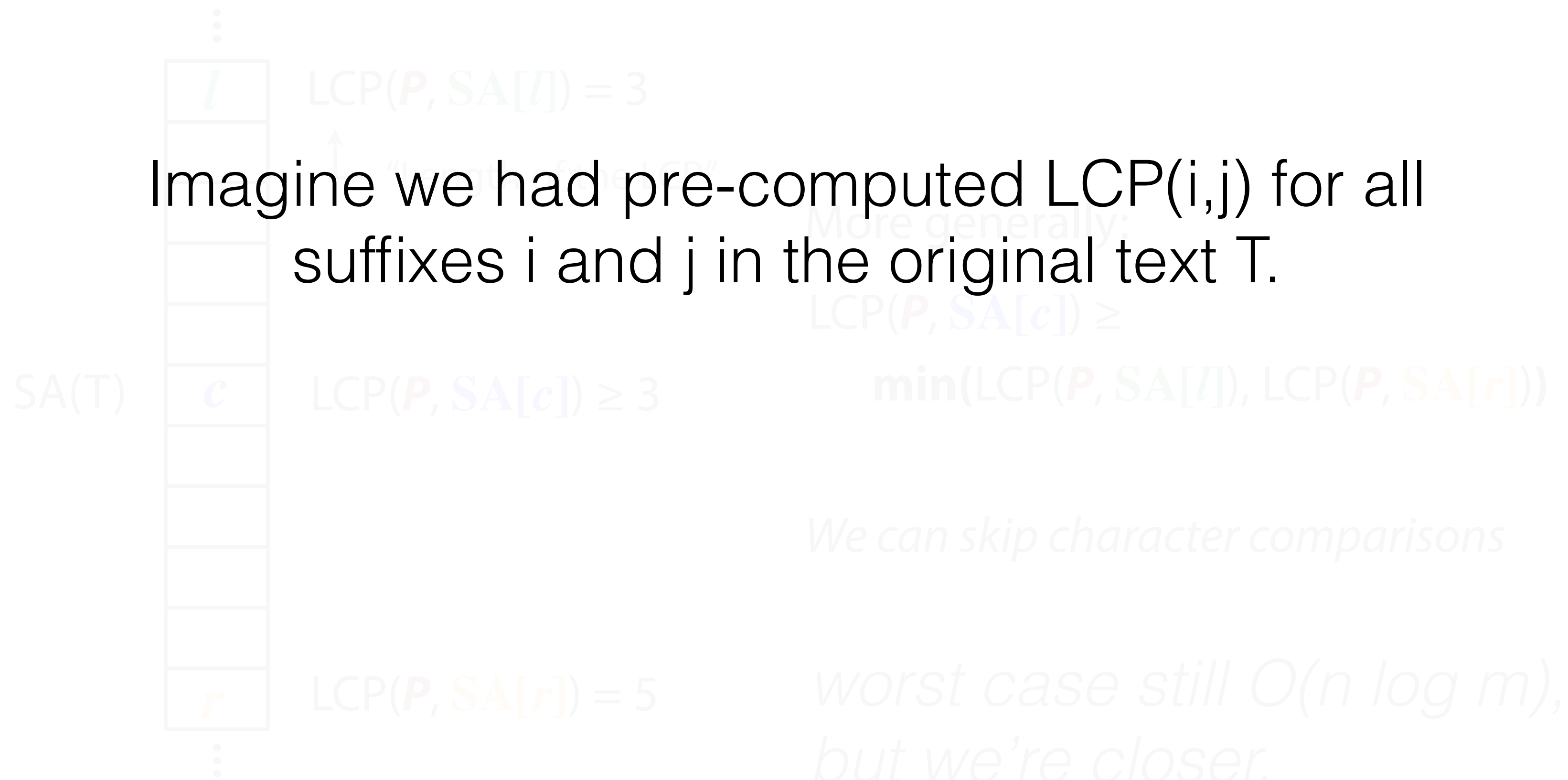
*worst case still $O(n \log m)$,
but we're closer.*

worst case example
 $S = ac_{M-2}b$, $P = c$

Suffix array: querying

Say we're comparing P to $SA[c]$ and we've already compared P to $SA[l]$ and $SA[r]$ in previous iterations.

Imagine we had pre-computed $LCP(i,j)$ for all suffixes i and j in the original text T .



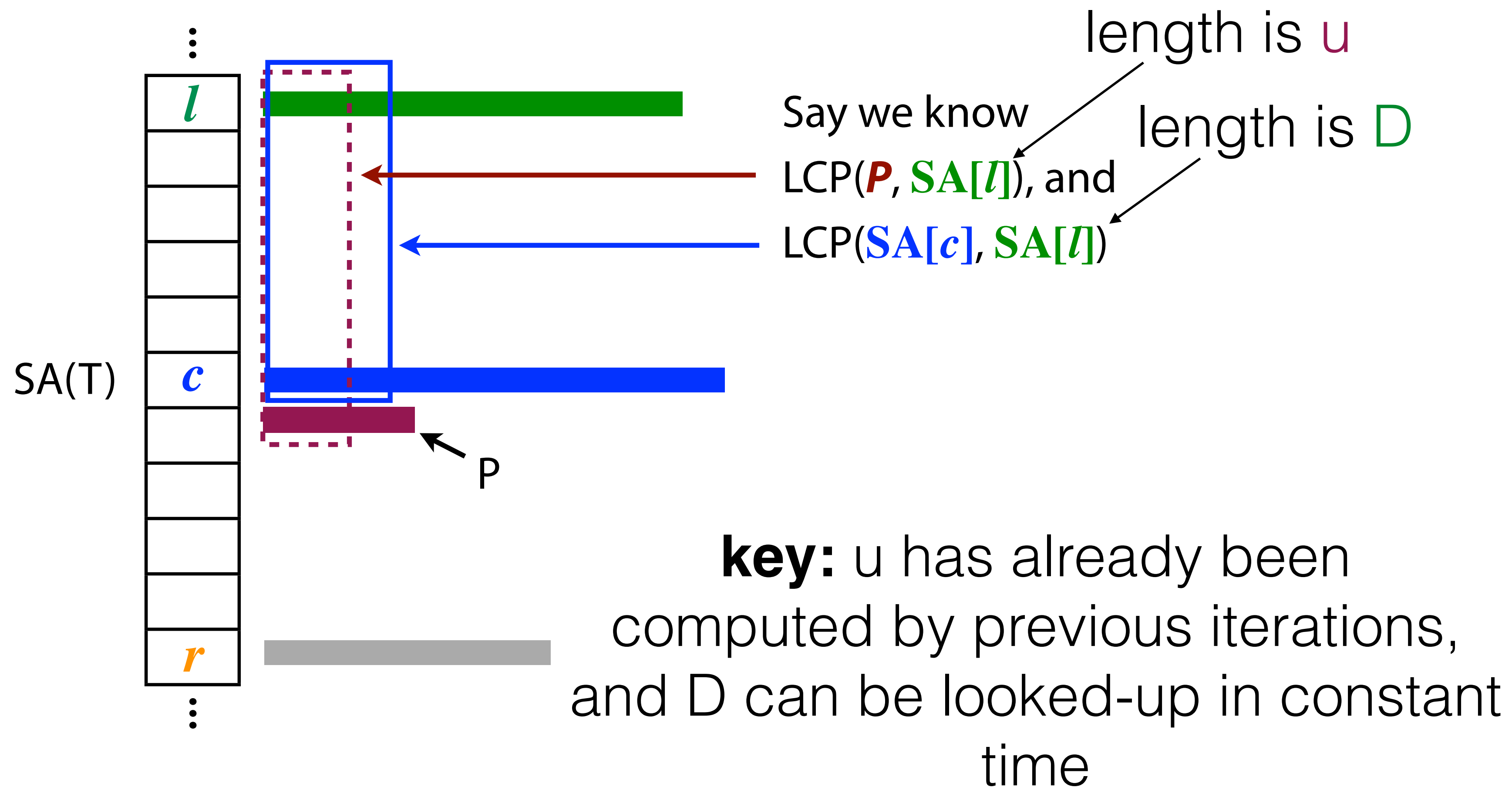
Suffix array: querying

Assume, wlog, that

$$D = \text{LCP}(\text{SA}[l], \text{SA}[c]) \geq D' = \text{LCP}(\text{SA}[c], \text{SA}[r])$$

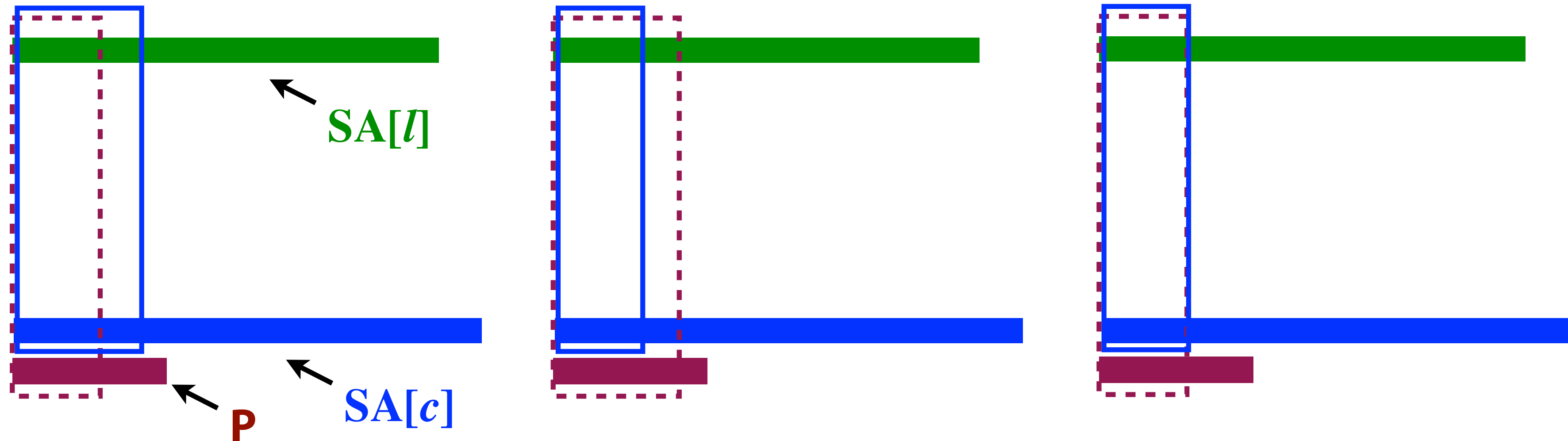
otherwise there are symmetric cases.

Take an iteration of binary search:



Suffix array: querying

Three cases: or, if $D' = \text{LCP}(\mathbf{P}, \text{SA}[r])$ is larger, 3 symmetric cases.



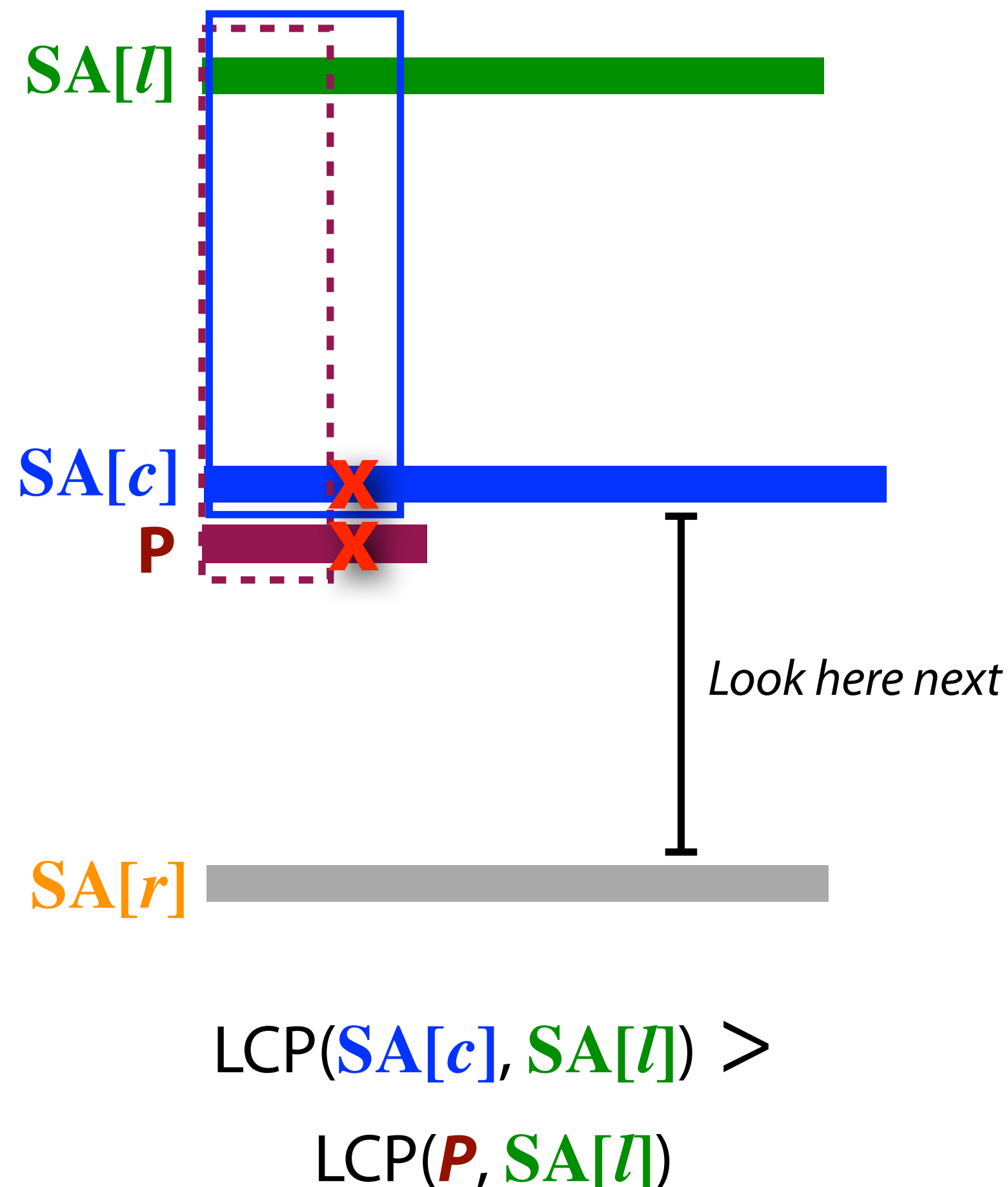
$$\text{LCP}(\text{SA}[c], \text{SA}[l]) > \\ \text{LCP}(\mathbf{P}, \text{SA}[l])$$

$$\text{LCP}(\text{SA}[c], \text{SA}[l]) < \\ \text{LCP}(\mathbf{P}, \text{SA}[l])$$

$$\text{LCP}(\text{SA}[c], \text{SA}[l]) = \\ \text{LCP}(\mathbf{P}, \text{SA}[l])$$

Suffix array: querying

Case 1: $\text{LCP}(\text{SA}[c], \text{SA}[l]) >$
 $\text{LCP}(\text{P}, \text{SA}[l])$



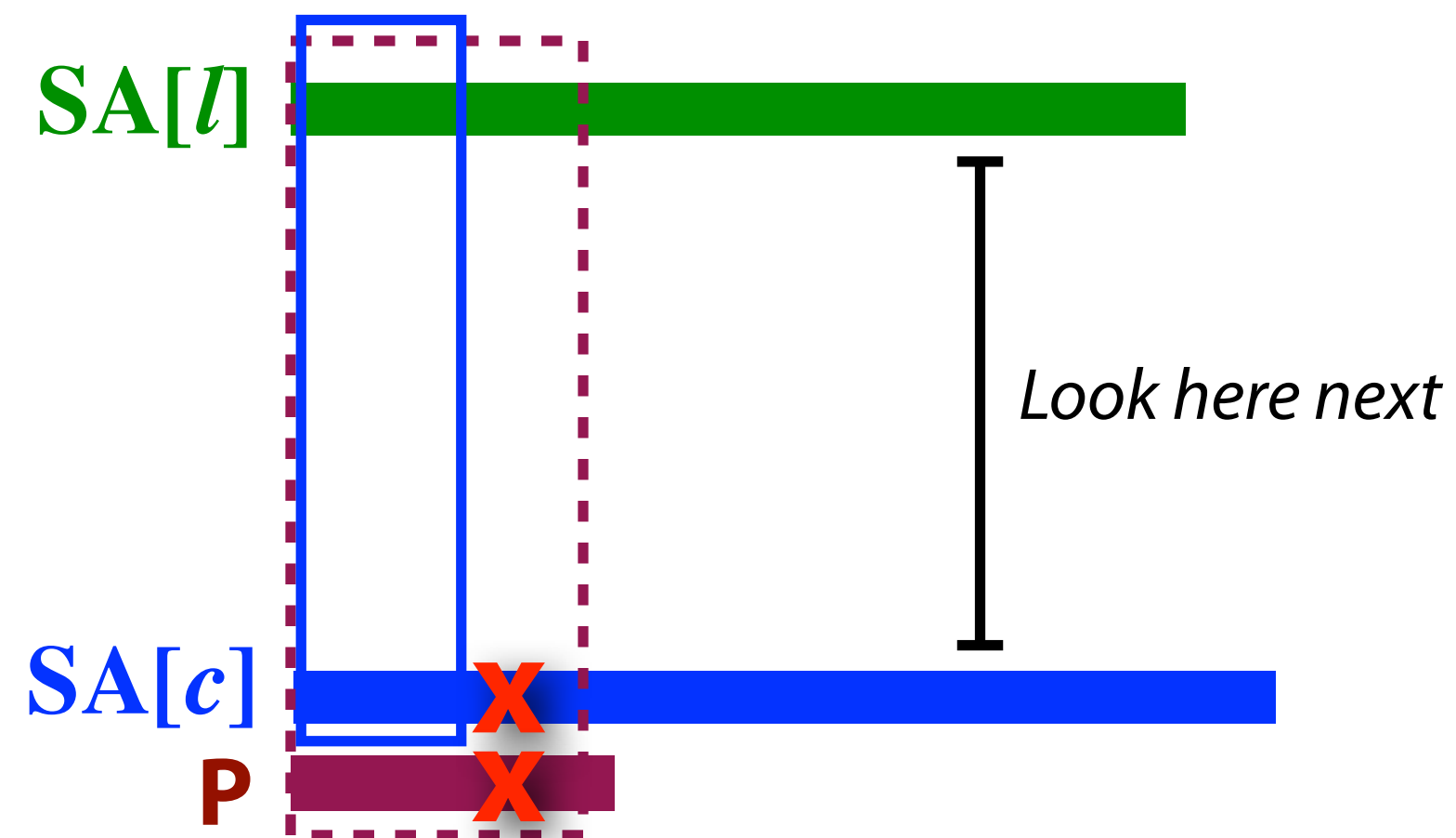
Next char of P after the $\text{LCP}(\text{P}, \text{SA}[l])$ must be *greater than* corresponding char of $\text{SA}[c]$

$$\text{P} > \text{SA}[c]$$

In this case, we compute $\text{LCP}(\text{P}[u:], \text{SA}[c][u:])$.
 c becomes our new l ,
and now we know the new $\text{LCP}(\text{P}, \text{SA}[l])$, b/c we just computed it!

Suffix array: querying

Case 2: $\text{LCP}(\text{SA}[c], \text{SA}[l]) < \text{LCP}(P, \text{SA}[l])$



Next char of $\text{SA}[c]$ after $\text{LCP}(\text{SA}[c], \text{SA}[l])$ must be *greater than* corresponding char of P

$$P < \text{SA}[c]$$

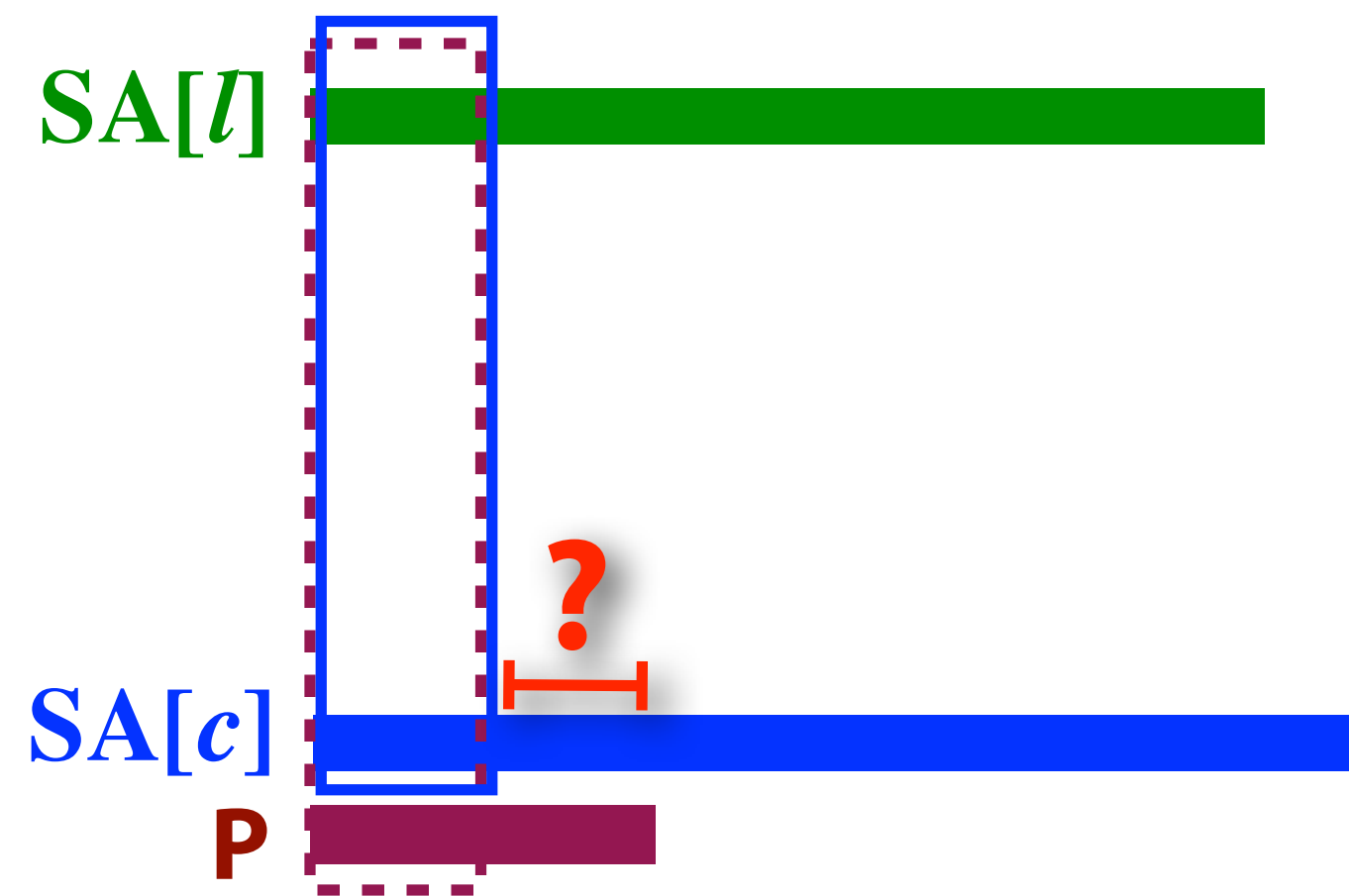
SA[r]

$$\text{LCP}(\text{SA}[c], \text{SA}[l]) < \text{LCP}(P, \text{SA}[l])$$

c becomes our new r,
and now we know the *new*
 $\text{LCP}(P, \text{SA}[r])$, b/c the LCP must
end right before the mismatch
position (i.e. must be = $\text{LCP}(\text{SA}[c], \text{SA}[l])$)!

Suffix array: querying

Case 3: $\text{LCP}(\text{SA}[c], \text{SA}[l]) =$
 $\text{LCP}(\text{P}, \text{SA}[l])$



Must do further character comparisons
between **P** and **SA[c]**

Each such comparison either:

- (a) mismatches, leading to a bisection
- (b) matches, in which case $\text{LCP}(\text{P}, \text{SA}[c])$ grows

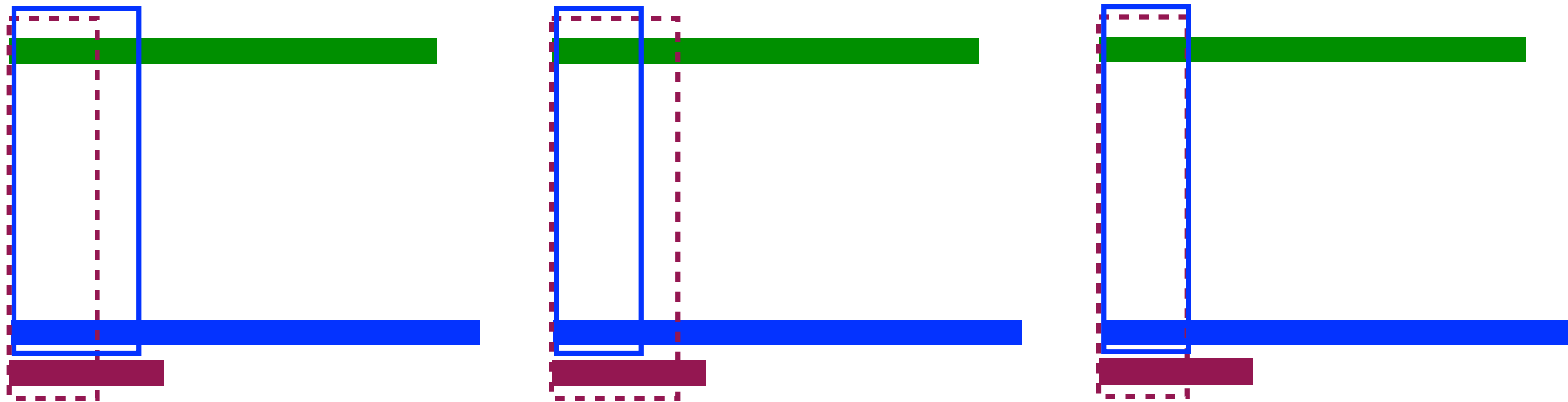
SA[r]

$\text{LCP}(\text{SA}[c], \text{SA}[l]) =$
 $\text{LCP}(\text{P}, \text{SA}[l])$

Suffix array: querying

We improved binary search on suffix array from $O(n \log m)$ to $O(n + \log m)$ using information about Longest Common Prefixes (LCPs).

LCPs between P and suffixes of T computed during search, LCPs *among* suffixes of T computed *offline*



$$\text{LCP}(\text{SA}[c], \text{SA}[l]) > \\ \text{LCP}(P, \text{SA}[l])$$

Bisect right!

$$\text{LCP}(\text{SA}[c], \text{SA}[l]) < \\ \text{LCP}(P, \text{SA}[l])$$

Bisect left!

$$\text{LCP}(\text{SA}[c], \text{SA}[l]) = \\ \text{LCP}(P, \text{SA}[l])$$

Compare some
characters, then bisect!

Sketch of Running Time

Thm. *Given the $LCP(X,Y)$ values, searching for a string P in a suffix array of length m now takes $O(|P| + \log m)$ time.*

In case 1 & 2, we make $O(1)$ comparisons and bisect left or right — there are at most $O(\log m)$ bisections.

In case 3 we try to match characters starting at some offset between $SA[c]$ and P . If they match, those characters will never be compared again, so there are at most $O(|P|)$ such comparisons.

Mismatching characters may be compared more than once.

But there can be only 1 mismatch / bisection. There are $O(\log m)$ bisections, so there are at most $O(\log m)$ mismatches.

$\therefore$ Total # of comparisons = $O(|P| + \log m)$.



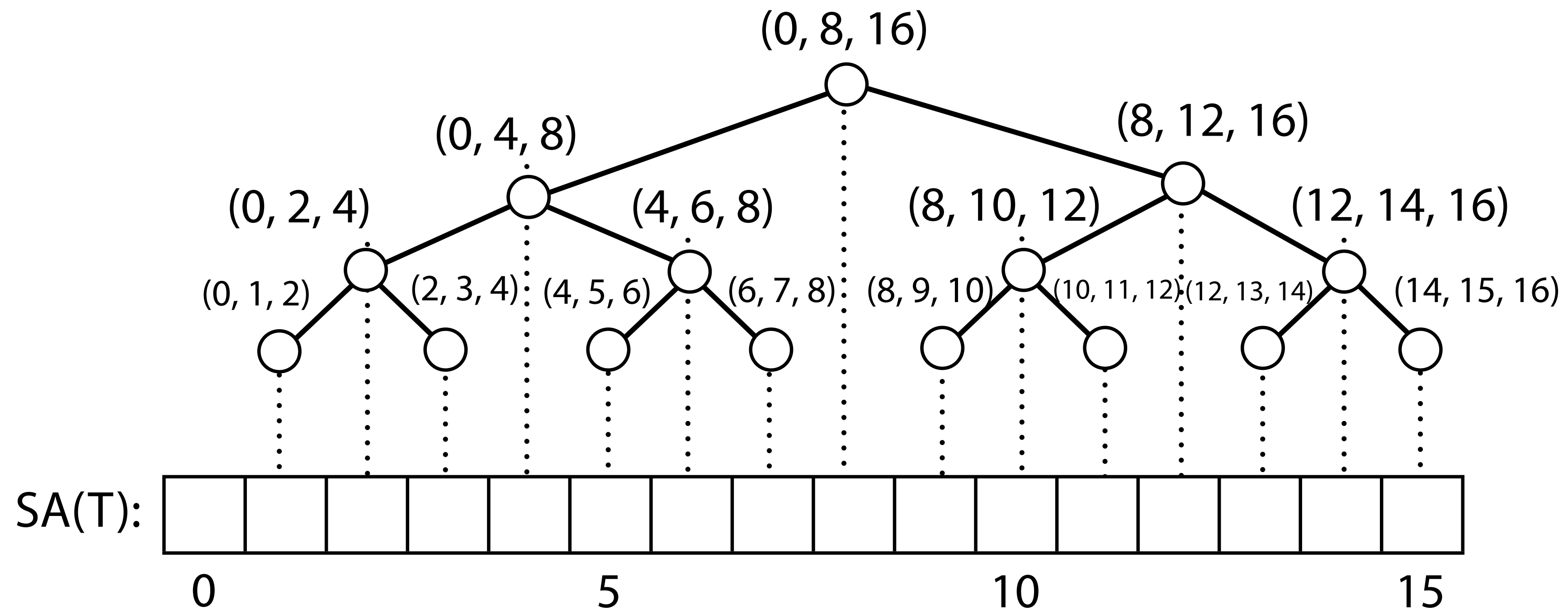
How to pre-compute LCP

- To perform this “efficient” search, we must be able to look up $\text{LCP}(\text{SA}[c], \text{SA}[l])$ and $\text{LCP}(\text{SA}[c], \text{SA}[r])$.
- How can we pre-compute this information *efficiently*?
 - Which LCP values do we need (*hint: not all of them*)?
 - Given LCP for left and right sub-interval of a search, how can we compute LCP for the containing interval?

Suffix array: LCPs

How to pre-calculate LCPs for every (l, c) and (c, r) pair in the search tree?

Triples are (l, c, r) triples

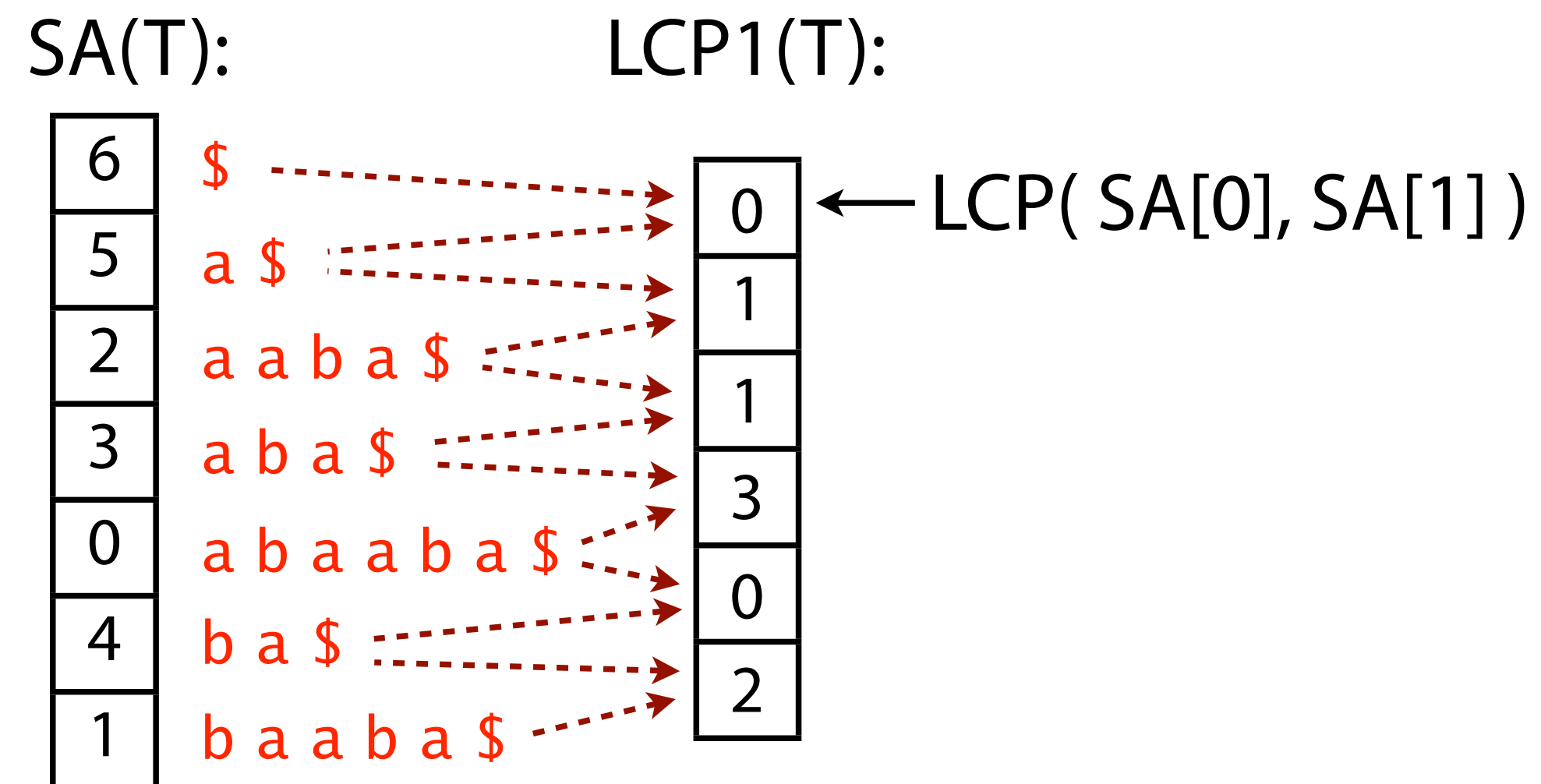


Example where $m = 16$ (incl. \$) # search tree nodes = $m - 1$

Suffix array: LCPs

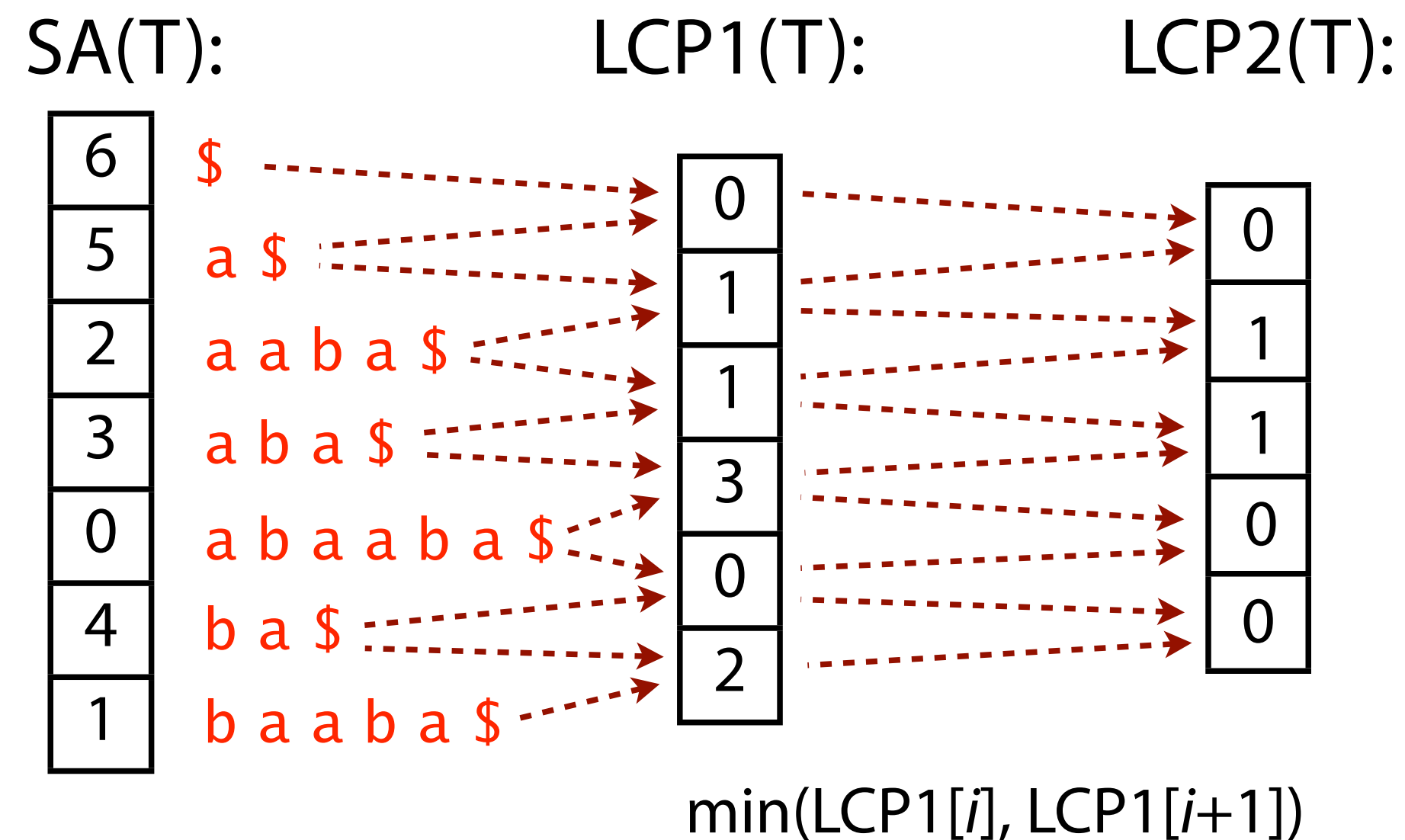
Suffix Array (SA) has m elements

Define LCP1 array with $m - 1$ elements such that $LCP[i] = LCP(SA[i], SA[i+1])$



Suffix array: LCPs

$$\text{LCP2}[i] = \text{LCP}(\text{SA}[i], \text{SA}[i+1], \text{SA}[i+2])$$

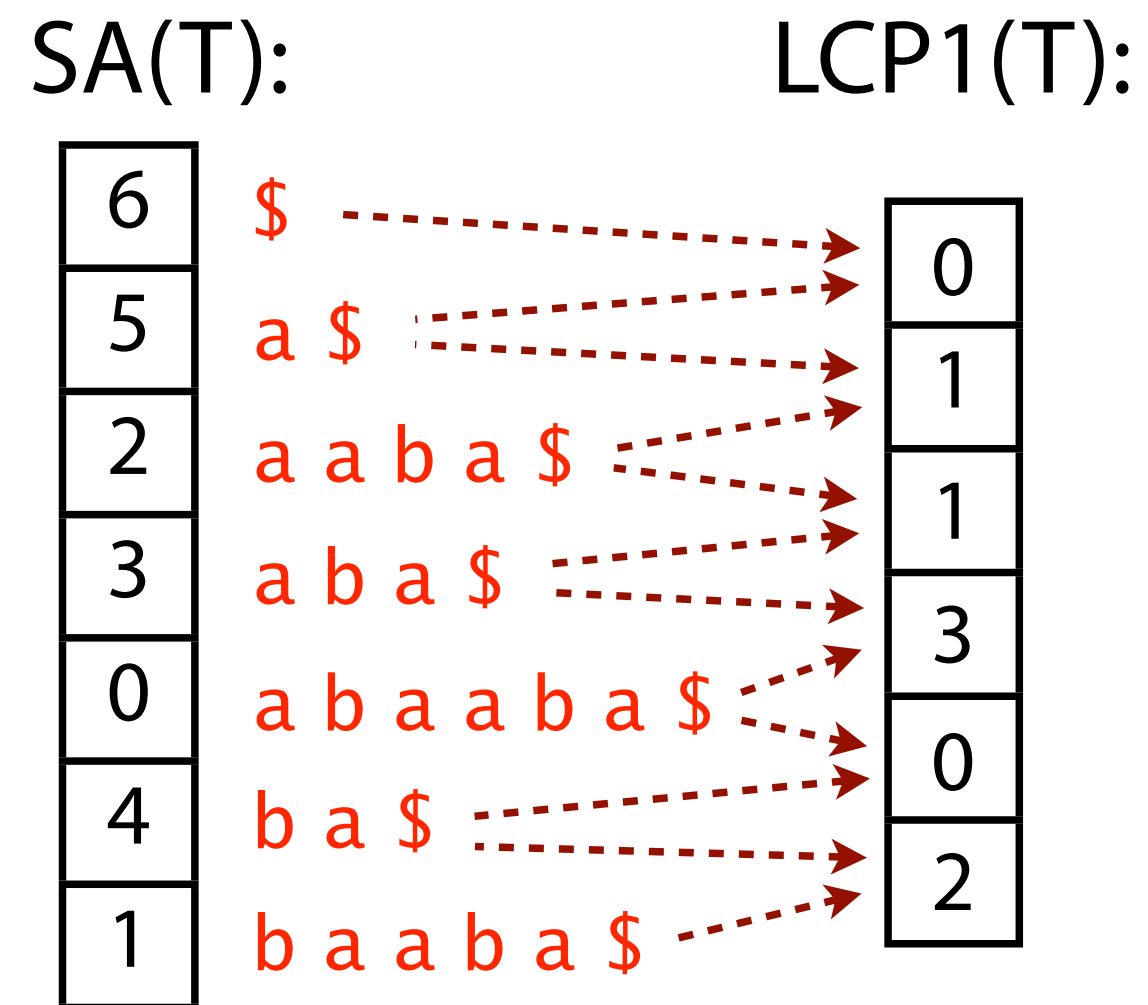


In fact, LCP of a range of consecutive suffixes in SA equals the minimum LCP1 among adjacent pairs in the range

LCP1 is a building block for other useful LCPs

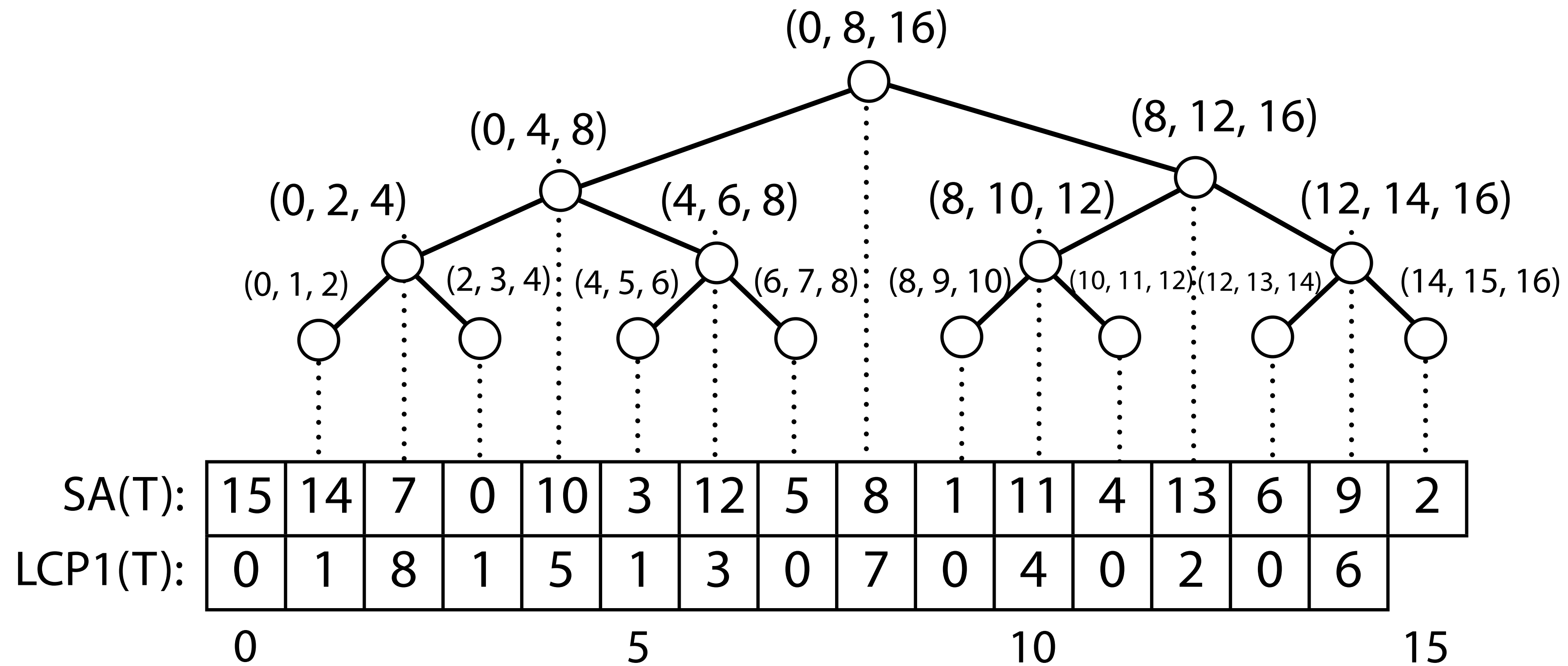
Suffix array: LCPs

Good time to calculate LCP1 it is *at the same time* as we *build* the suffix array, since putting the suffixes in order involves breaking ties after common prefixes



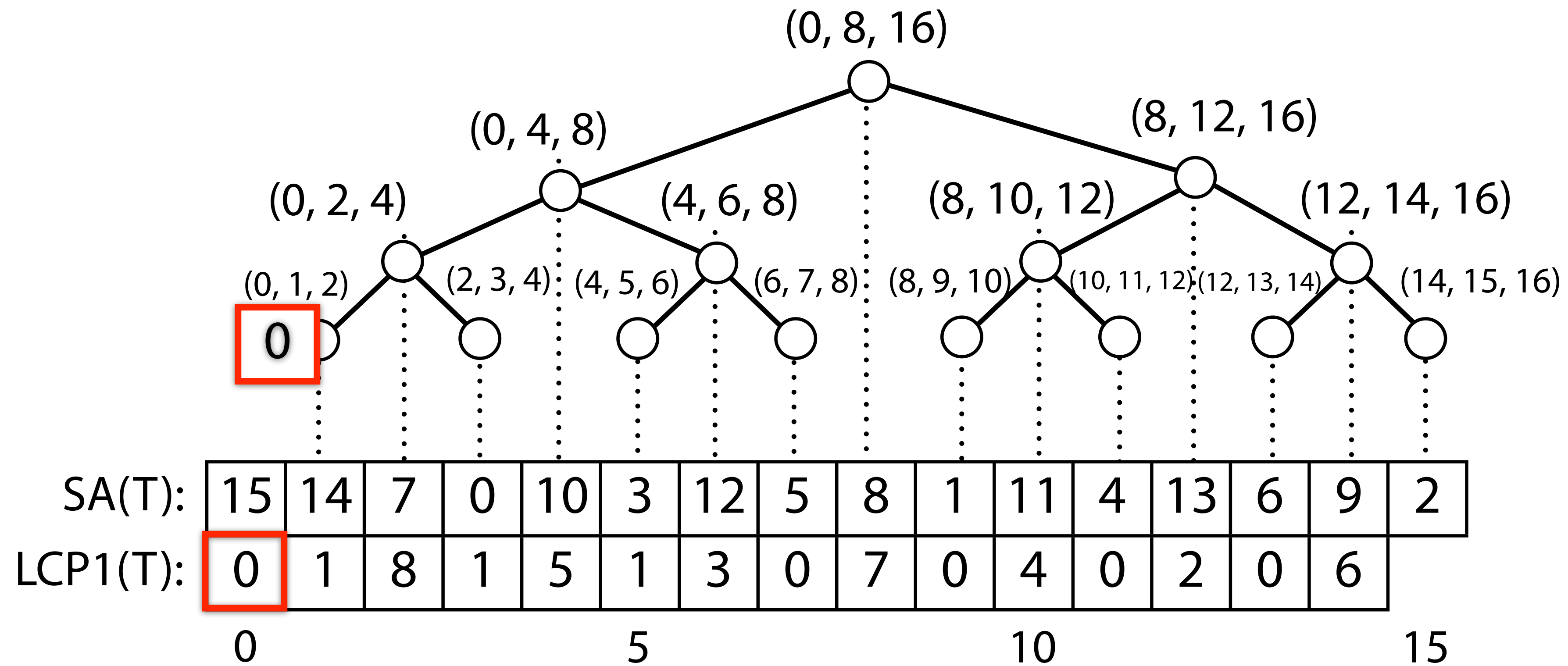
Suffix array: LCPs

T = abracadabracada



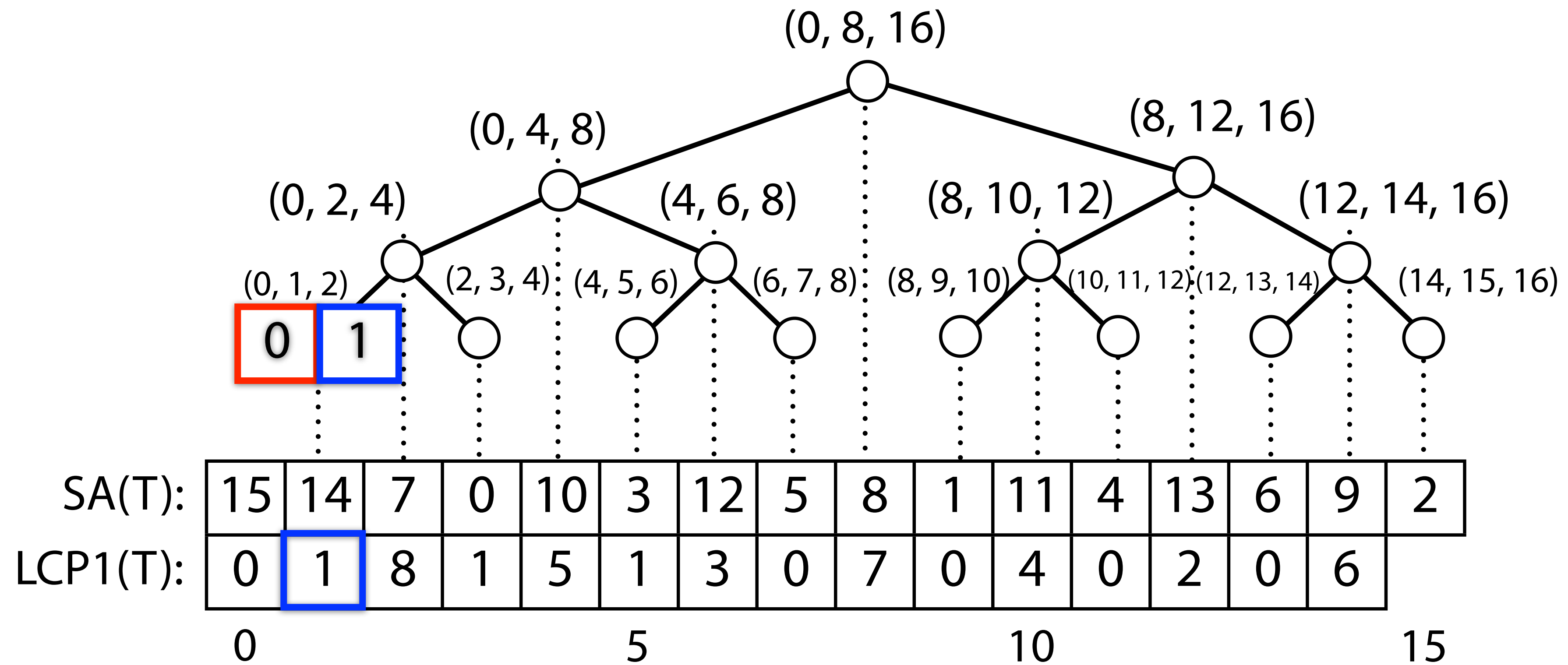
Suffix array: LCPs

T = abracadabracada\$



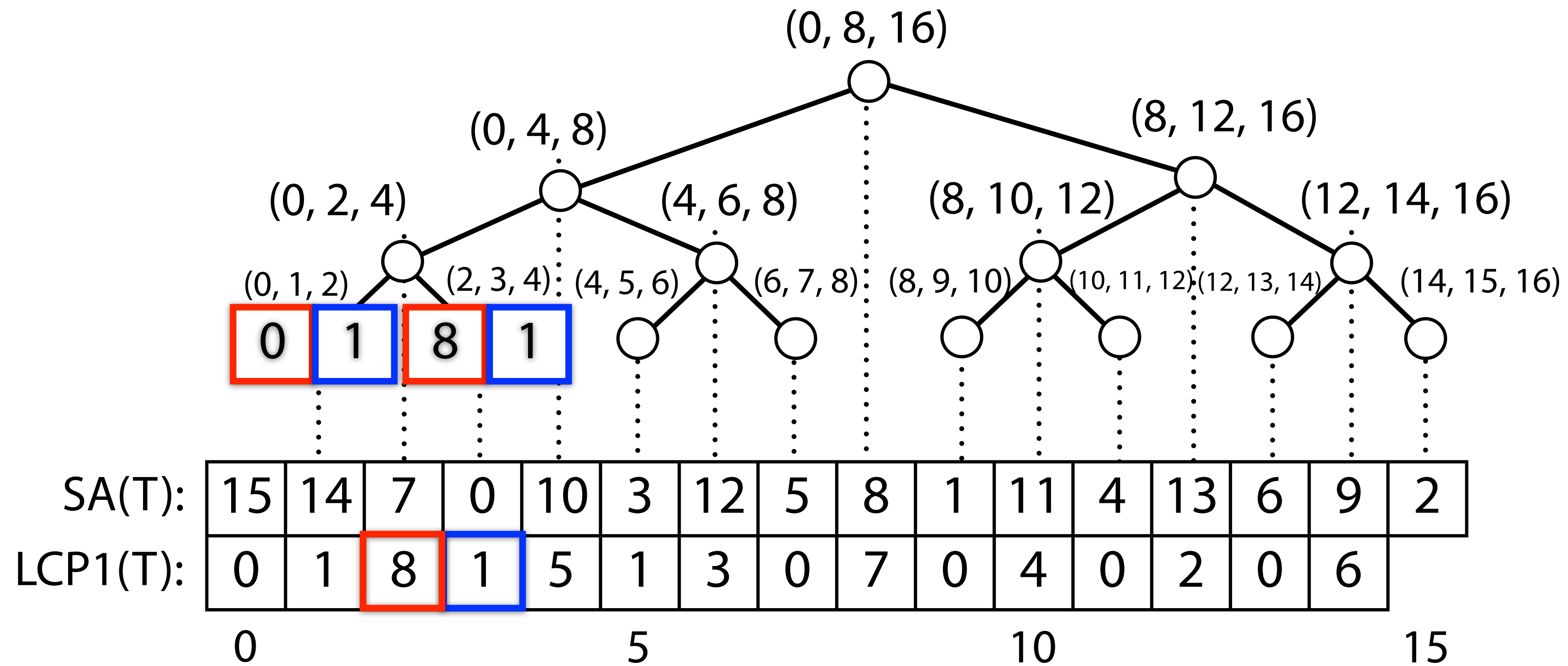
Suffix array: LCPs

T = abracadabracada\$



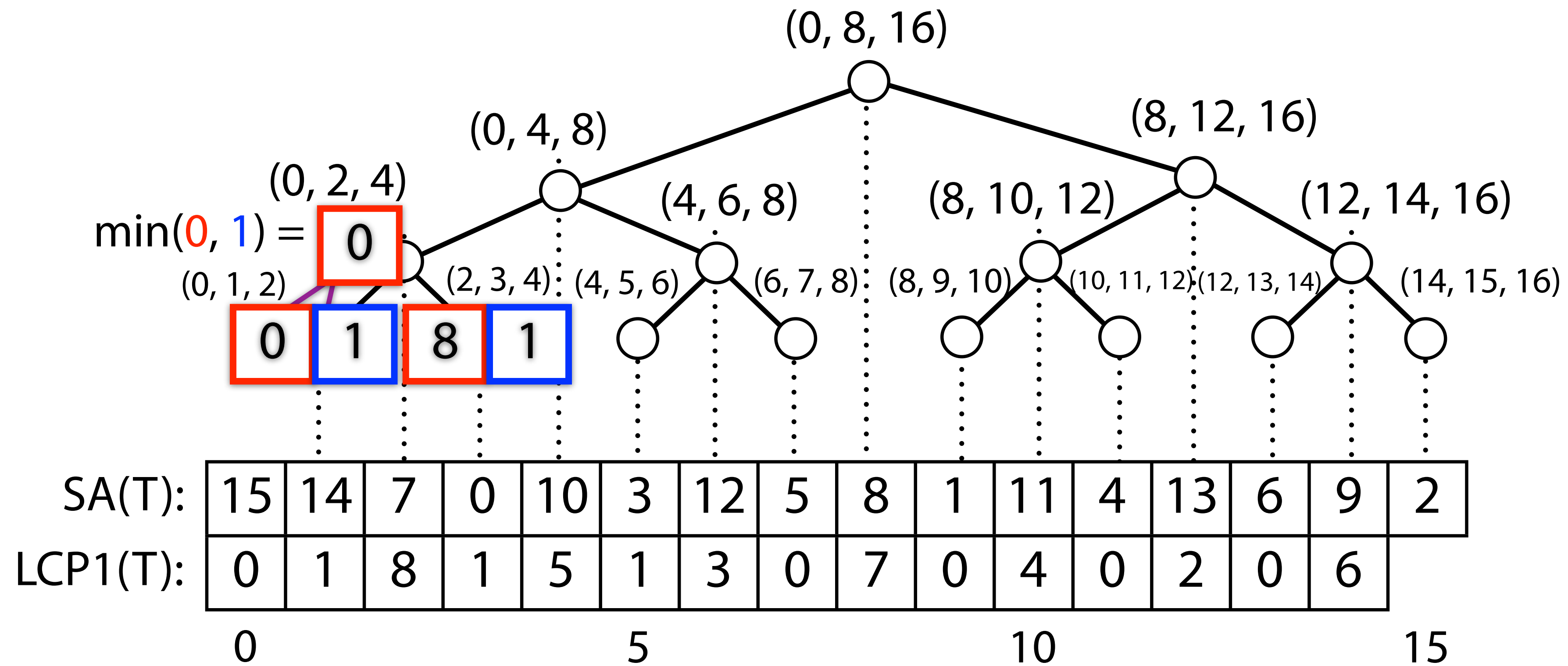
Suffix array: LCPs

T = abracadabracada\$



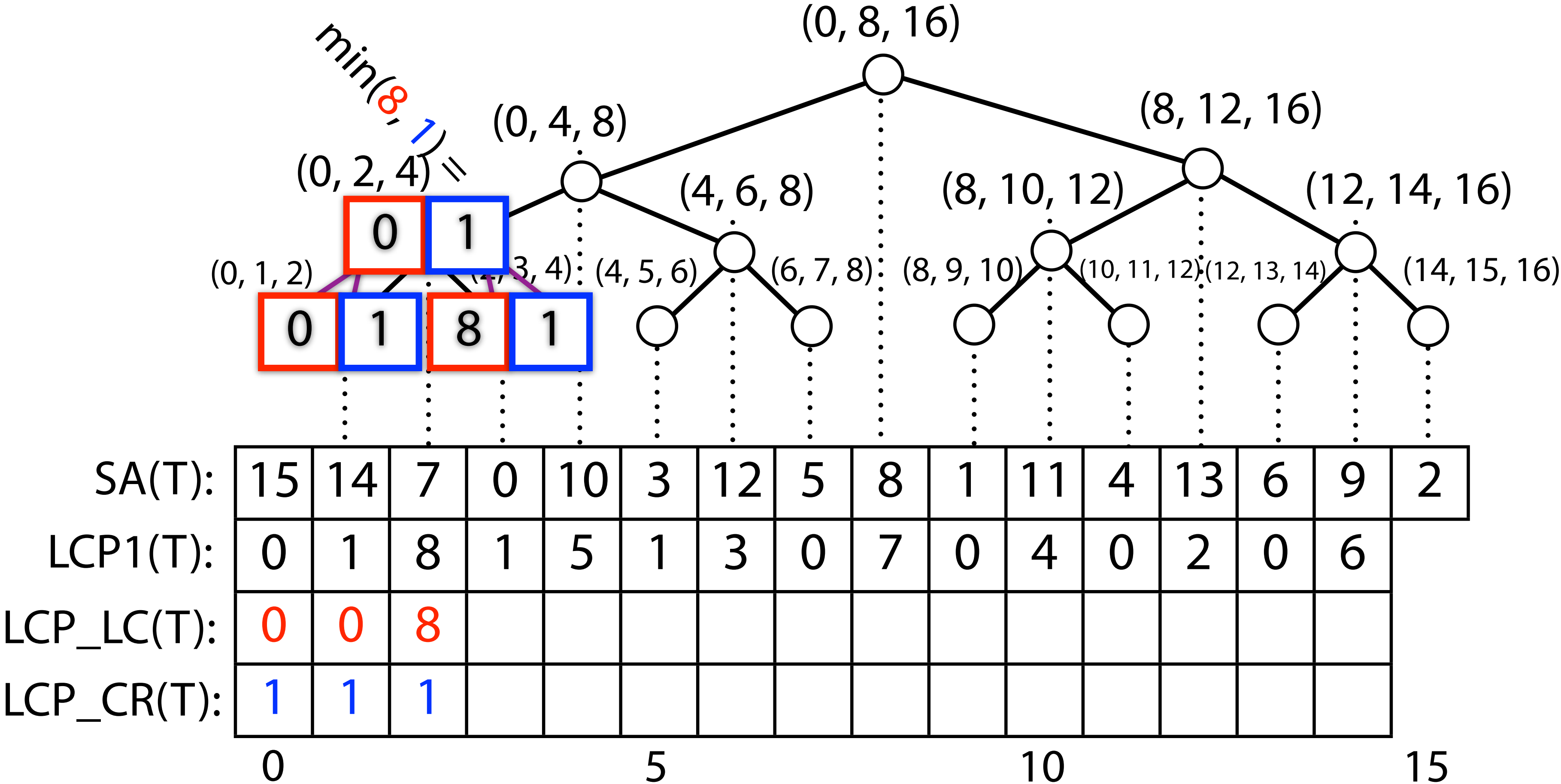
Suffix array: LCPs

T = abracadabracada\$



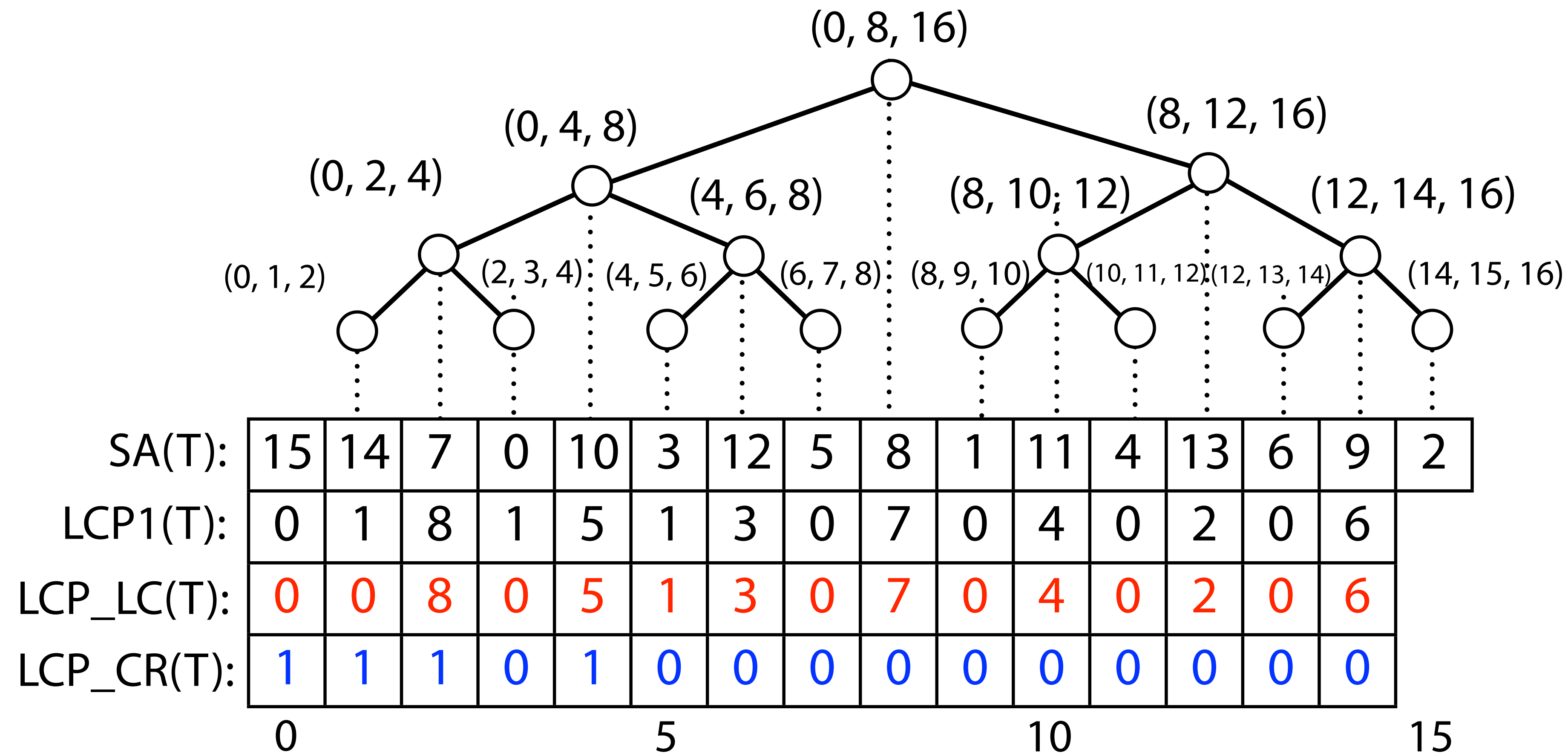
Suffix array: LCPs

T = abracadabracada\$



Suffix array: LCPs

T = abracadabracada\$



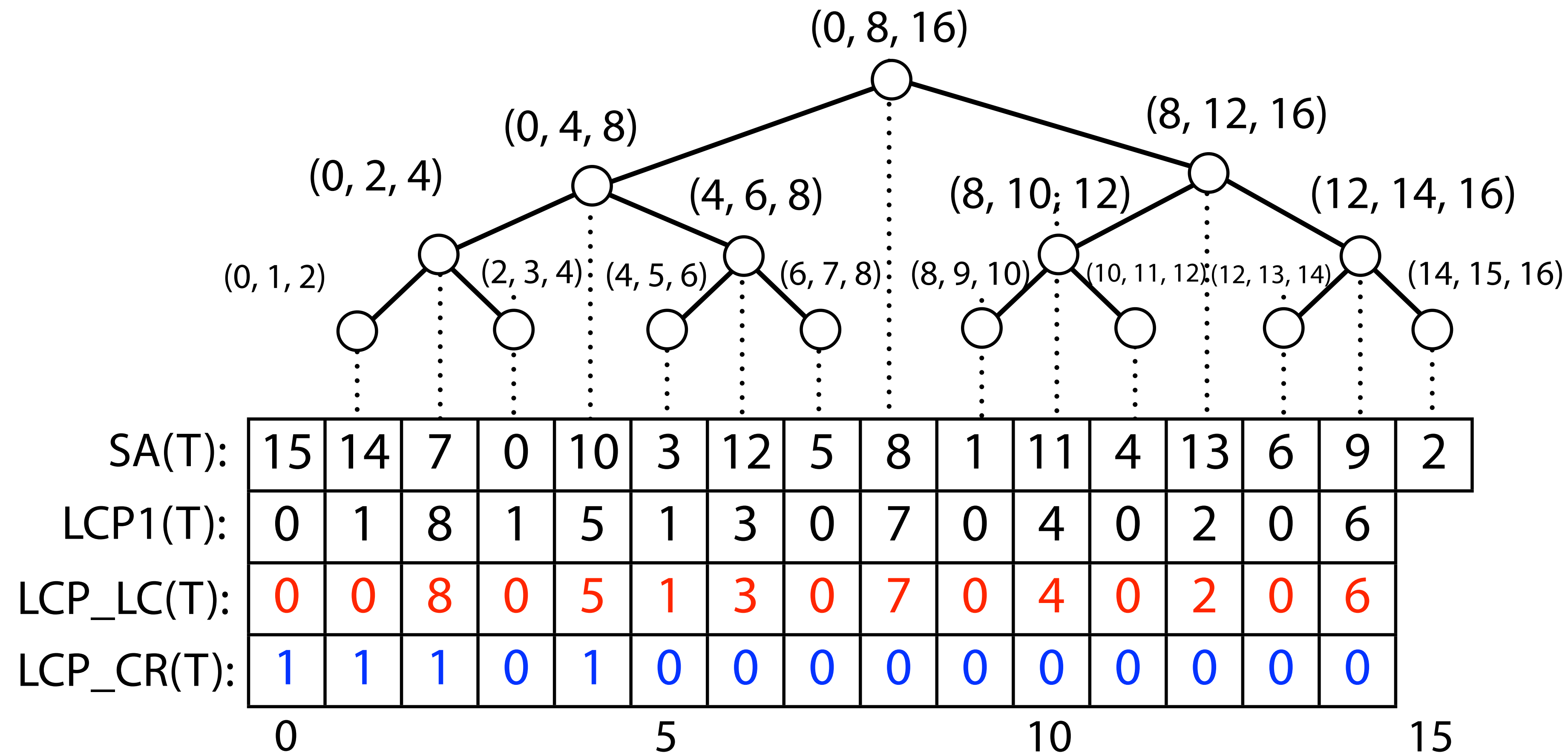
NOTE: These arrays are “shifted” by 1 — the value in LCP_LC corresponding to (0, 1, 2) is at LCP_LC[0], not LCP_LC[1]. So, to look up LCP(SA[l], SA[c]) we look at LCP_LC[c-1]

Suffix array: LCPs

Can be done in:

$O(m)$ time and space

$T = \text{abracadabracada\$}$



NOTE: These arrays are “shifted” by 1 — the value in LCP_LC corresponding to $(0, 1, 2)$ is at $\text{LCP_LC}[0]$, not $\text{LCP_LC}[1]$. So, to look up $\text{LCP}(\text{SA}[l], \text{SA}[c])$ we look at $\text{LCP_LC}[c-1]$

Suffix array: querying review

We saw 3 ways to query (binary search) the suffix array:

1. Typical binary search. Ignores LCPs. $O(n \log m)$.
2. Binary search with some skipping using LCPs between P and T 's suffixes. Still $O(n \log m)$, but it can be argued it's near $O(n + \log m)$ in practice. Gusfield: "Simple Accelerant"
3. Binary search with skipping using all LCPs, including LCPs among T 's suffixes. $O(n + \log m)$. Gusfield: "Super Accelerant"

How much space do they require?

1. $\sim m$ integers (SA)
2. $\sim m$ integers (SA)
3. $\sim 3m$ integers (SA, LCP_LC, LCP_CR)

Suffix array: performance comparison

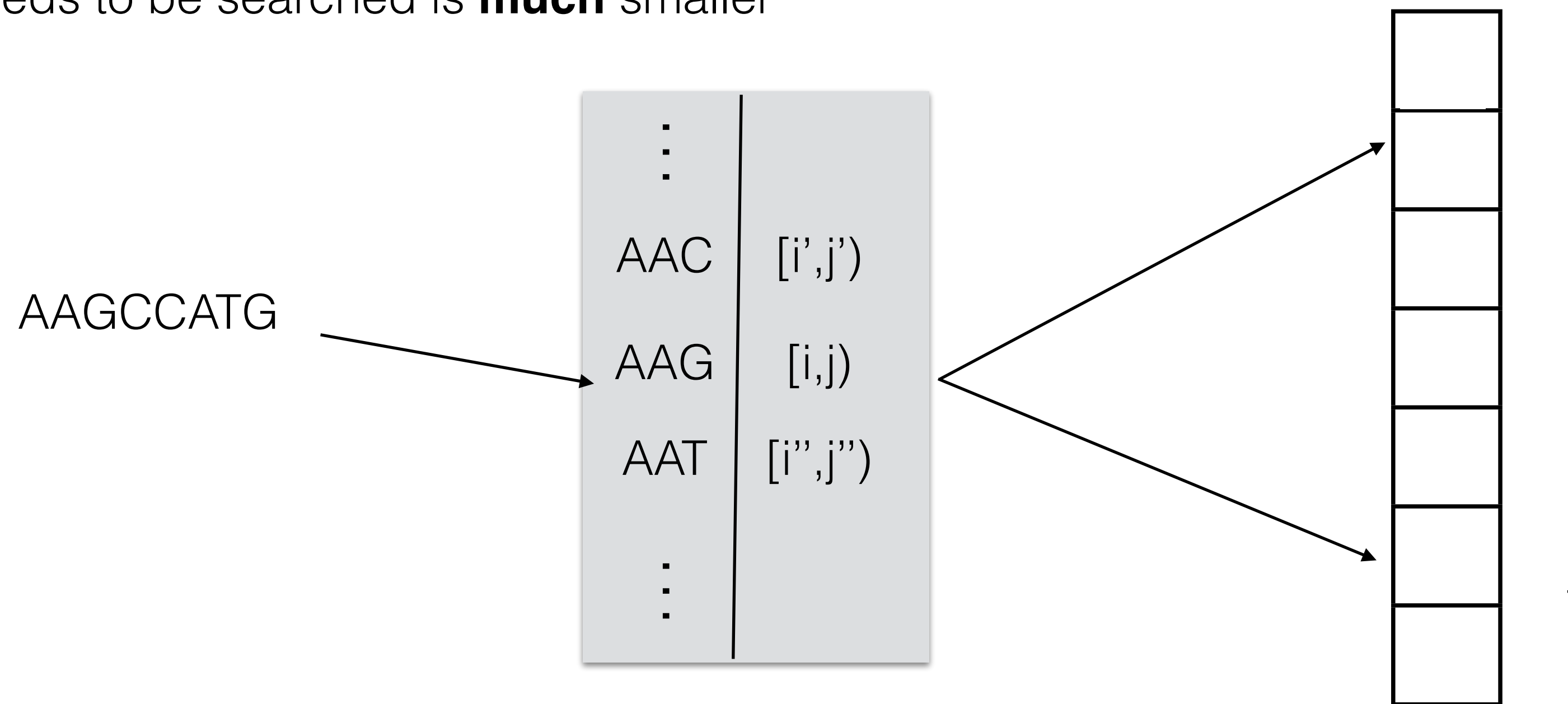
	Super accelerant	Simple accelerant	No accelerant
python -O	68.78 s	69.80 s	102.71 s
pypy -O	5.37 s	5.21 s	8.74 s
# character comparisons	99.5 M	117 M	235 M

Matching 500K 100-nt substrings to the ~ 5 million nt-long *E. coli* genome. Substrings drawn randomly from the genome.

Index building time not included

Another “practical” speedup

- Imagine you will never search for patterns of length $< k$ (e.g. 4-mers are non-informative in any moderately-sized genome)
- Consider the following “enhanced” suffix array:
 - Build a hash-table from k -mers to suffix array intervals. Now, any pattern of length $k' > k$ must start with some hashed prefix of length k . Generally, the interval that needs to be searched is **much** smaller



Now, you only need to search the interval $[i, j)$ — $O(n * \log(j-i))$ time

Can provide considerable speedup

	dna	english	proteins	sources	xml
$m = 16$					
SA	1.00	1.00	1.00	1.00	1.00
SA-LUT2	1.13	1.34	1.36	1.43	1.35
SA-LUT3	1.17	1.49	1.61	1.65	1.47
SA-hash	3.75	2.88	2.70	2.90	2.03
$m = 64$					
SA	1.00	1.00	1.00	1.00	1.00
SA-LUT2	1.12	1.33	1.34	1.42	1.34
SA-LUT3	1.17	1.49	1.58	1.64	1.44
SA-hash	3.81	2.87	2.62	2.75	1.79

k=12
Linear probing
Hash at $\alpha=0.5$

Table 1. Speedups with regard to the search speed of the plain suffix array, for the five datasets and pattern lengths $m = 16$ and $m = 64$

Some other clever ideas#:

- Use a k-ary (B-tree) layout
- Use a lookup table where keys are concatenated Huffman codes of fixed bit length
- Use alternative strategy (doubling/gallop) to find the right SA boundary

“Two Simple Full-Text Indexes Based on the Suffix Array”, Szymon Grabowski and Marcin Raniszewski

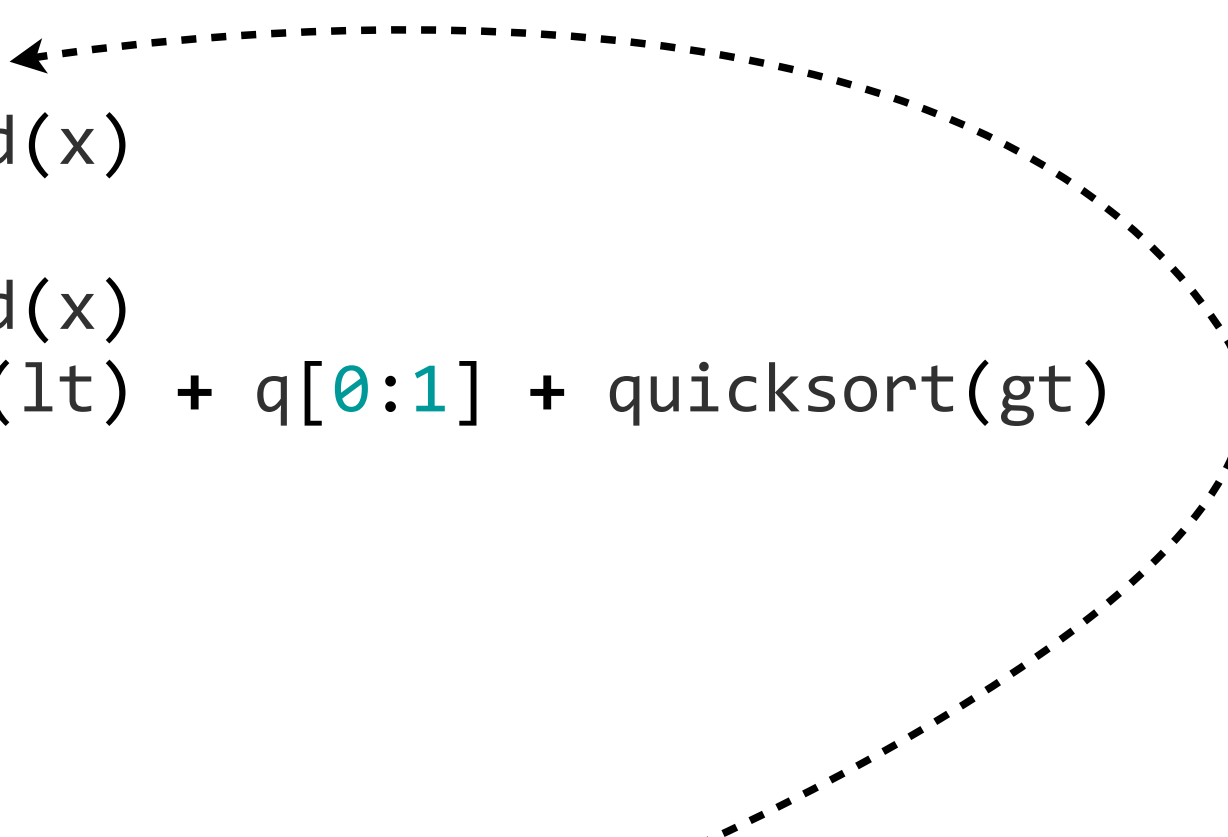
Kowalski, Tomasz, et al. "Suffix arrays with a twist." *arXiv preprint arXiv:1607.08176* (2016).

Suffix array: sorting suffixes

One idea: Use your favorite sort, e.g., quicksort

0	a b a a b a \$
1	b a a b a \$
2	a a b a \$
3	a b a \$
4	b a \$
5	a \$
6	\$

```
def quicksort(q):  
    lt, gt = [], []  
    if len(q) <= 1:  
        return q  
    for x in q[1:]:  
        if x < q[0]:  
            lt.append(x)  
        else:  
            gt.append(x)  
    return quicksort(lt) + q[0:1] + quicksort(gt)
```



Expected time: $O(m^2 \log m)$

Not $O(m \log m)$ because a suffix comparison is $O(m)$ time

Suffix array: sorting suffixes

One idea: Use a sort algorithm that's aware that the items being sorted are strings, e.g. "multikey quicksort"

0	a b a a b a \$
1	b a a b a \$
2	a a b a \$
3	a b a \$
4	b a \$
5	a \$
6	\$

Essentially $O(m^2)$ time

Bentley, Jon L., and Robert Sedgwick. "Fast algorithms for sorting and searching strings." *Proceedings of the eighth annual ACM-SIAM symposium on Discrete algorithms*. Society for Industrial and Applied Mathematics, 1997

Suffix array: sorting suffixes

Another idea: Use a sort algorithm that's aware that the items being sorted are all suffixes of the same string

Original suffix array paper suggested an $O(m \log m)$ algorithm

Manber U, Myers G. "Suffix arrays: a new method for on-line string searches." SIAM Journal on Computing 22.5 (1993): 935-948.

Other popular $O(m \log m)$ algorithms have been suggested

Larsson NJ, Sadakane K. Faster suffix sorting. Technical Report LU-CS-TR: 99-214, LUNDFD6/(NFCS-3140)/1-43/(1999), Department of Computer Science, Lund University, Sweden, 1999.

More recently $O(m)$ algorithms have been demonstrated!

Kärkkäinen J, Sanders P. "Simple linear work suffix array construction." Automata, Languages and Programming (2003): 187-187.

Ko P, Aluru S. "Space efficient linear time construction of suffix arrays." *Combinatorial Pattern Matching*. Springer Berlin Heidelberg, 2003.

And there are comparable advances with respect to LCP1

The Skew Algorithm (aka DC3)

Kärkkäinen & Sanders, 2003

- **Main idea: Divide suffixes into 3 groups:**
 - Those starting at positions $i=0,3,6,9,\dots$ ($i \bmod 3 = 0$)
 - Those starting at positions $1,4,7,10,\dots$ ($i \bmod 3 = 1$)
 - Those starting at positions $2,5,8,11,\dots$ ($i \bmod 3 = 2$)
- For simplicity, assume text length is a multiple of 3 after padding with a special character.

$T[0, n) =$

0	1	2	3	4	5	6	7	8	9	10	11
y	a	b	b	a	d	a	b	b	a	d	o

...

$SA = (12, 1, 6, 4, 9, 3, 8, 2, 7, 5, 10, 11, 0)$

Basic Outline:

- Recursively handle suffixes from the $i \bmod 3 = 1$ and $i \bmod 3 = 2$ groups.
- Merge the $i \bmod 3 = 0$ group at the end.

*slide courtesy of Carl Kingsford

Step 0 — Constructing a sample

These are called the “sample suffixes”

Step 0: Construct a sample. For $k = 0, 1, 2$, define

$$B_k = \{i \in [0, n] \mid i \bmod 3 = k\}.$$

Let $C = B_1 \cup B_2$ be the set of *sample positions* and S_C the set of *sample suffixes*.

Example. $B_1 = \{1, 4, 7, 10\}$, $B_2 = \{2, 5, 8, 11\}$, i.e., $C = \{1, 4, 7, 10, 2, 5, 8, 11\}$.

Step I — Sorting the sample

Step 1: Sort sample suffixes. For $k = 1, 2$, construct the strings

$$R_k = [t_k t_{k+1} t_{k+2}] [t_{k+3} t_{k+4} t_{k+5}] \dots [t_{\max B_k} t_{\max B_k+1} t_{\max B_k+2}]$$

whose characters are triples $[t_i t_{i+1} t_{i+2}]$. Note that the last character of R_k is always unique because $t_{\max B_k+2} = 0$. Let $R = R_1 \odot R_2$ be the concatenation of R_1 and R_2 . Then the (nonempty) suffixes of R correspond to the set S_C of sample suffixes: $[t_i t_{i+1} t_{i+2}] [t_{i+3} t_{i+4} t_{i+5}] \dots$ corresponds to S_i . The correspondence is order preserving, i.e., by sorting the suffixes of R we get the order of the sample suffixes S_C .

Example. $R = [\text{abb}][\text{ada}][\text{bba}][\text{do0}][\text{bba}][\text{dab}][\text{bad}][\text{o00}]$.

Step I — Sorting the sample

To sort the suffixes of R , first radix sort the characters of R and rename them with their ranks to obtain the string R' . If all characters are different, the order of characters gives directly the order of suffixes. Otherwise, sort the suffixes of R' using Algorithm DC3.

Example. $R' = (1, 2, 4, 6, 4, 5, 3, 7)$ and $SA_{R'} = (8, 0, 1, 6, 4, 2, 5, 3, 7)$.

Step 1.5 — Sorting the sample

Example. $R = [\text{abb}][\text{ada}][\text{bba}][\text{do0}][\text{bba}][\text{dab}][\text{bad}][\text{o00}]$.

Once the sample suffixes are sorted, assign a rank to each suffix. For $i \in C$, let $\text{rank}(S_i)$ denote the rank of S_i in the sample set S_C . Additionally, define $\text{rank}(S_{n+1}) = \text{rank}(S_{n+2}) = 0$. For $i \in B_0$, $\text{rank}(S_i)$ is undefined.

Example. $\text{rank}(S_i)$

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
$\text{rank}(S_i)$	$\perp$	1	4	$\perp$	2	6	$\perp$	5	3	$\perp$	7	8	$\perp$	0	0

Note: In this example, we won't be able to get all of these ranks until the algorithm has recursed a second time.

R' is ranks after first round / recursion. [000][abb][ada][bad][bba][bba][dab][do0][o00]

Example. $R' = (1, 2, 4, 6, 4, 5, 3, 7)$ and $SA_{R'} = (8, 0, 1, 6, 4, 2, 5, 3, 7)$.

1 group

2 group

Step 1.5 — Sorting the sample

Once the sample suffixes are sorted, assign a rank to each suffix. For $i \in C$, let $rank(S_i)$ denote the rank of S_i in the sample set S_C . Additionally, define $rank(S_{n+1}) = rank(S_{n+2}) = 0$. For $i \in B_0$, $rank(S_i)$ is undefined.

$$T[0, n) = \begin{matrix} & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 \\ y & a & b & b & a & d & a & b & b & a & d & o \end{matrix}$$

Example. $R = \overbrace{[abb][ada][bba][do0]}^{1 \bmod 3 \text{ triplets}} \overbrace{[bba][dab][bad][o00]}^{2 \bmod 3 \text{ triplets}}.$

$$\begin{matrix} i & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 \\ \text{Eg. rank}(S_i) & \perp & 1 & 4 & \perp & 2 & 5 & \perp & 4 & 3 & \perp & 6 & 7 & \perp \end{matrix}$$

Step 1.5 — Sorting the sample

Once the sample suffixes are sorted, assign a rank to each suffix. For $i \in C$, let $rank(S_i)$ denote the rank of S_i in the sample set S_C . Additionally, define $rank(S_{n+1}) = rank(S_{n+2}) = 0$. For $i \in B_0$, $rank(S_i)$ is undefined.

Note: After only 1 level of recursion, these suffixes would be “tied”

Example. $R = [abb][ada][bba][do0][bba][dab][bad][o00]$.



Step 1.5 — Sorting the sample

$T[0, n) =$

0	1	2	3	4	5	6	7	8	9	10	11
y	a	b	b	a	d	a	b	b	a	d	o

Example. $R =$

1	2	4	7	4	6	3	8	
[abb]	[ada]	[bba]	[do0]	[bba]	[dab]	[bad]	[o00]	...

$R_2 = [247][474][463][638][800] \dots$

These suffixes were tied at the previous level, but here, we can resolve them. The *lexical renaming* allows us to compare longer and longer suffixes of the text.

The ranks for original B₁ B₂ samples

Example. $\begin{array}{c} i \\ \text{rank}(S_i) \end{array} \begin{array}{cccccccccccccccc} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 \\ \perp & 1 & 4 & \perp & 2 & 6 & \perp & 5 & 3 & \perp & 7 & 8 & \perp & 0 & 0 \end{array}$

The resolved ranks here represent what we'd get after a second level of recursion.

Step 2 — Sorting the non-sample suffixes

Step 2: Sort nonsample suffixes. Represent each nonsample suffix $S_i \in S_{B_0}$ with the pair $(t_i, \text{rank}(S_{i+1}))$. Note that $\text{rank}(S_{i+1})$ is always defined for $i \in B_0$. Clearly we have, for all $i, j \in B_0$,

$$S_i \leq S_j \iff (t_i, \text{rank}(S_{i+1})) \leq (t_j, \text{rank}(S_{j+1})).$$

The pairs $(t_i, \text{rank}(S_{i+1}))$ are then radix sorted.

Example. $S_{12} < S_6 < S_9 < S_3 < S_0$ because $(0, 0) < (\mathbf{a}, 5) < (\mathbf{a}, 7) < (\mathbf{b}, 2) < (\mathbf{y}, 1)$.

Step 2 — Sorting the non-sample suffixes

Step 3: Merge. The two sorted sets of suffixes are merged using a standard comparison-based merging. To compare suffix $S_i \in S_C$ with $S_j \in S_{B_0}$, we distinguish two cases:

$$\begin{aligned} i \in B_1 : \quad S_i \leq S_j &\iff (t_i, \overset{S_{B1} \downarrow}{\text{rank}}(S_{i+1})) \leq (t_j, \overset{S_{B0} \downarrow}{\text{rank}}(S_{j+1})) \\ i \in B_2 : \quad S_i \leq S_j &\iff (t_i, \overset{S_{B2} \downarrow}{\text{rank}}(S_{i+2})) \leq (t_j, \overset{S_{B1} \downarrow}{\text{rank}}(S_{j+2})) \end{aligned}$$

$\overset{S_{B2} \uparrow}{}, \overset{S_{B0} \uparrow}{\phantom{t_{i+1}}}, \overset{S_{B1} \uparrow}{\phantom{\text{rank}(S_{i+2})}} \leq \overset{S_{B0} \uparrow}{}, \overset{S_{B1} \uparrow}{\phantom{t_{j+1}}}, \overset{S_{B2} \uparrow}{\phantom{\text{rank}(S_{j+2})}}$

Note that the ranks are defined in all cases.

Example. $S_1 < S_6$ because $(\mathbf{a}, 4) < (\mathbf{a}, 5)$ and $S_3 < S_8$ because $(\mathbf{b}, \mathbf{a}, 6) < (\mathbf{b}, \mathbf{a}, 7)$.

Running Time

$$T(n) = O(n) + T(2n/3)$$

time to sort and merge

array in recursive calls
is 2/3rds the size of
starting array

Solves to $T(n) = O(n)$:

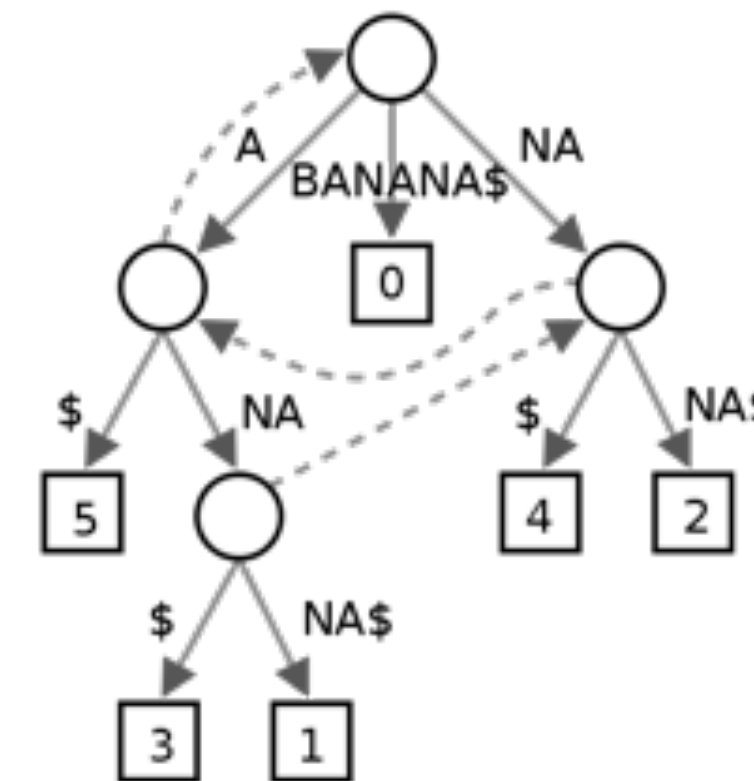
- Expand big-O notation: $T(n) \leq cn + T(2n/3)$ for some c .
- Guess: $T(n) \leq 3cn$
- Induction step: assume that is true for all $i < n$.
- $T(n) \leq cn + 3c(2n/3) = cn + 2cn = 3cn \quad \square$

Suffix array: summary

Suffix array gives us index that is:

(a) Just m integers, with $O(n \log m)$ worst-case query time, but close to $O(n + \log m)$ in practice

or **(b)** $3m$ integers, with $O(n + \log m)$ worst case



Suffix Tree

6	\$
5	A\$
3	ANA\$
1	ANANA\$
0	BANANA\$
4	NA\$
2	NANA\$

Suffix Array

(a) will often be preferable: index for entire human genome fits in ~12 GB instead of > 45 GB

Enhanced Suffix Arrays

Abouelhoda, Mohamed Ibrahim, Stefan Kurtz, and Enno Ohlebusch. "**Replacing suffix trees with enhanced suffix arrays.**" Journal of Discrete Algorithms 2.1 (2004): 53-86.

Can restore the **full** asymptotic efficiency of suffix trees with a small number of auxiliary tables.

Application	Enhanced suffix array					
	suftab $4n$ bytes	lcptab n bytes	childtab n bytes	suflink $2n$ bytes	S $n \log \Sigma $ bits	bwttab $n \log \Sigma $ bits
<i>esasupermax</i>	✓	✓				✓
<i>esamum</i>	✓	✓				✓
<i>esarep</i>	✓	✓				✓
Ziv–Lempel	✓	✓				
<i>esamatch</i>	✓	✓	✓		✓	
shortest unique sub.	✓	✓	✓			
<i>esams</i>	✓	✓	✓	✓	✓	

The operations that can be done optimally in an enhanced suffix array (esa), and the aux. tables required for them.

Many Suffix Array Variants

Compressed suffix arrays⁺ — require even less space

Compressed enhanced suffix arrays[✦] — strive for the best of both worlds and allow interesting query times like $O(n \log|\Sigma| + k)$ for finding k occurrences of a pattern, where $|\Sigma|$ is the size of the alphabet (independent of m).

⁺R. Grossi and J. S. Vitter, Compressed Suffix Arrays and Suffix Trees, with Applications to Text Indexing and String Matching, SIAM Journal on Computing, 35(2), 2005, 378-407

[✦]Ohlebusch, Enno, and Simon Gog. "A compressed enhanced suffix array supporting fast string matching." String Processing and Information Retrieval. Springer Berlin Heidelberg, 2009. ^{*}

Massively parallel construction

RESEARCH

Open Access

Fast, parallel, and cache-friendly suffix array construction



Jamshed Khan^{1*}, Tobias Rubel¹, Erin Molloy¹, Laxman Dhulipala¹ and Rob Patro^{1*}

Abstract

Purpose String indexes such as the suffix array (SA) and the closely related longest common prefix (LCP) array are fundamental objects in bioinformatics and have a wide variety of applications. Despite their importance in practice, few scalable parallel algorithms for constructing these are known, and the existing algorithms can be highly non-trivial to implement and parallelize.

Methods In this paper we present *CAPS-SA*, a simple and scalable parallel algorithm for constructing these string indexes inspired by samplesort and utilizing an LCP-informed mergesort. Due to its design, *CAPS-SA* has excellent memory-locality and thus incurs fewer cache misses and achieves strong performance on modern multicore systems with deep cache hierarchies.

Results We show that despite its simple design, *CAPS-SA* outperforms existing state-of-the-art parallel SA and LCP-array construction algorithms on modern hardware. Finally, motivated by applications in modern aligners where the query strings have bounded lengths, we introduce the notion of a bounded-context SA and show that *CAPS-SA* can easily be extended to exploit this structure to obtain further speedups. We make our code publicly available at <https://github.com/jamshed/CaPS-SA>.

Keywords Suffix array, Longest common prefix, Data structures, Indexing, Parallel algorithms

Massively parallel construction

Key idea is very simple:

Sample sort (partitions handled in parallel)

Construct LCP1 during sort, use LCPs to speed up merge

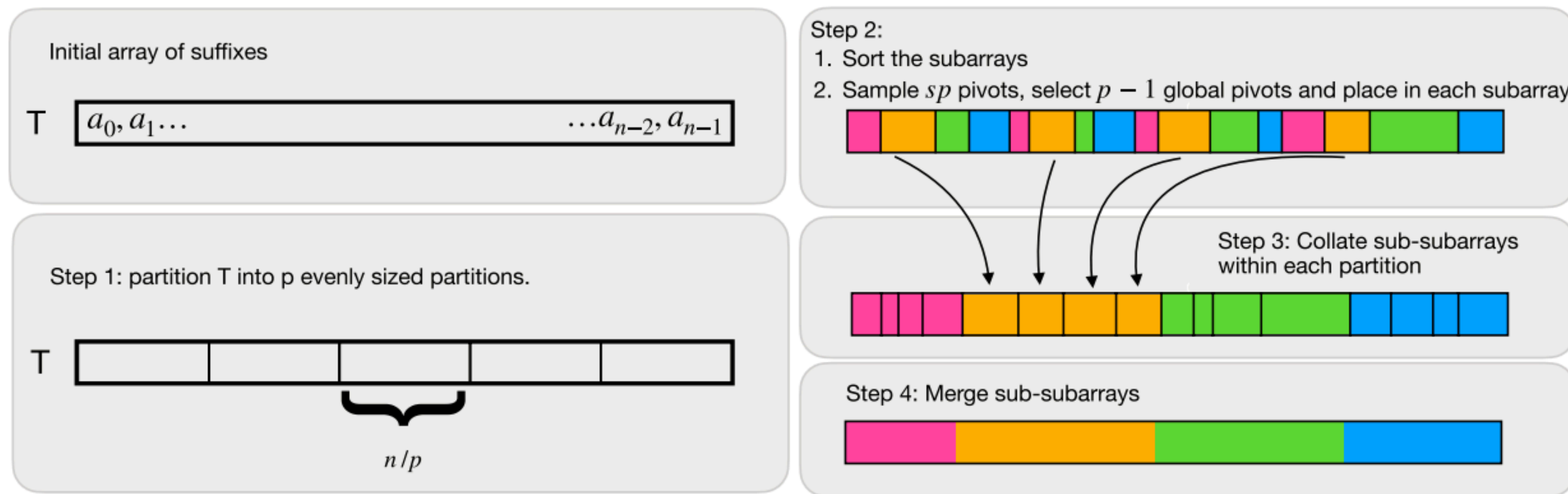


Fig. 1 Overview of CAPS-SA. In the first step of the algorithm the input text T is partitioned evenly across p partitions. Then each partition is sorted, pivots are sampled using the sampling routine, and located within each partition to create sub-partitions. Subsequently each sub-partition is collated. Finally the MERGE routine is used to complete the suffix and the LCP-array construction

LCP-enhanced merging

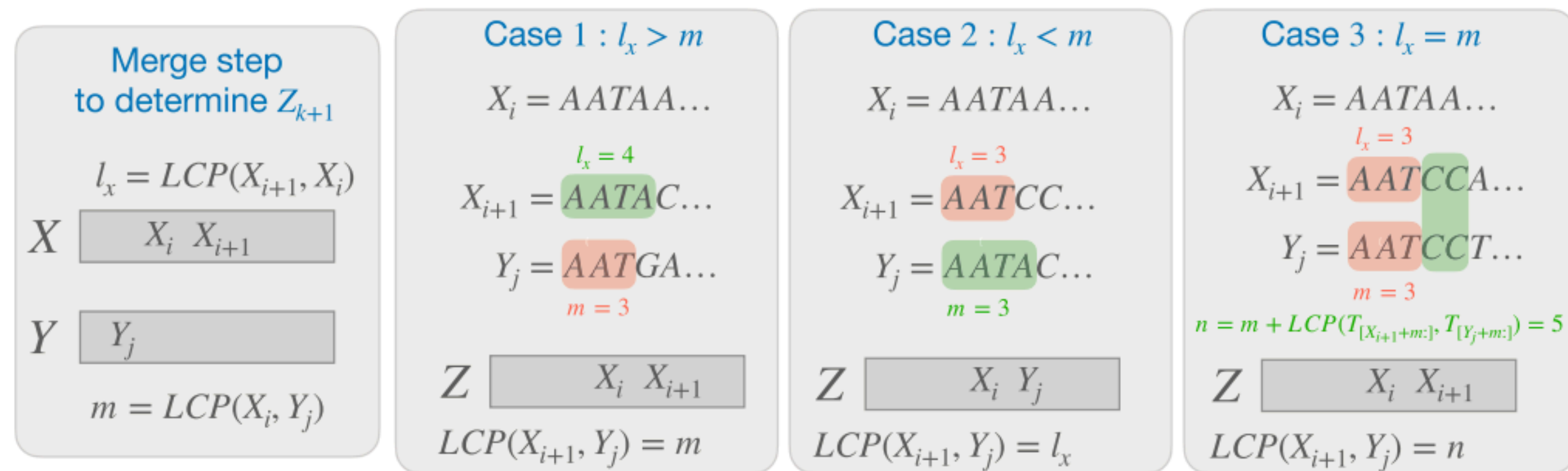


Fig. 2 Figure illustrating the cases that can occur on the $(k + 1)$ th step of the `MERGE` routine, which determines Z_{k+1} . Cases 1 and 2 require $\mathcal{O}(1)$ work and simply compare the LCP-lengths of the previous step and $LCP(X_{i+1}, X_i)$, which are already available. Step 3 requires work proportional to $\mathcal{O}(LCP(X_{i+1}, Y_j) - m)$, since X_{i+1}, Y_j already share a prefix of size m

```

MERGE( $X, Y, L_X, L_Y, Z, L_Z, T$ )
    // merges the sorted arrays of suffixes (of  $T$ )  $X$ 
    // and  $Y$  with LCP arrays  $L_X$  and  $L_Y$  resp.,
    // into  $Z$ ; also constructs  $Z$ 's LCP array in  $L_Z$ 
1   $i = j = k = 0$ 
2   $m = 0$  // LCP of the last compared pair
3   $l_x = 0$  //  $LCP(X_i, X_{i-1})$ 
4  while  $i < |X|$  and  $j < |Y|$ 
5       $l_x = L_{X_i}$ 
6      if  $l_x > m$ 
7           $Z_k = X_i, L_{Z_k} = l_x, m = m$ 
8      elseif  $l_x < m$ 
9           $Z_k = Y_j, L_{Z_k} = m, m = l_x$ 
10     else
11          $n = m + LCP(T_{[X_i+m:]}, T_{[Y_j+m:]})$ 
12          $Z_k = \begin{cases} X_i & \text{if } T_{X_i+n} < T_{Y_j+n} \\ Y_j & \text{otherwise} \end{cases}$ 
13          $L_{Z_k} = \begin{cases} l_x & \text{if } Z_k == X_i \\ m & \text{otherwise} \end{cases}$ 
14          $m = n$ 
15     if  $Z_k == X_i$ 
16          $i = i + 1$ 
17     else
18          $j = j + 1$ 
19          $SWAP(X, Y), SWAP(L_X, L_Y), SWAP(i, j)$ 
20      $k = k + 1$ 
21      $COPY(X_{[i:]}, Z_{[k:]})$ ,  $COPY(L_{X_{[i:]}} L_{Z_{[k:]}})$ 
22 if  $j < |Y|$ 
23      $Z_k = Y_j, L_{Z_k} = m$ 
24      $COPY(Y_{[j+1:]}, Z_{[k+1:]})$ ,
     $COPY(L_{Y_{[j+1:]}} L_{Z_{[k+1:]}})$ 

```


Runtime

Theorem 3 *The total number of symbol comparisons made across all the MERGE-SORT and MERGE-PARTITION instances in CAPS-SA for an n -length text is $\mathcal{O}(n \log n + \sum_{i=1}^{n-1} L_i)$ whp, where L is the output LCP-array.*

Worst-case runtime is dominated by LCP computation
(when there are many long LCPs)

CaPS-SA scales well with thread count

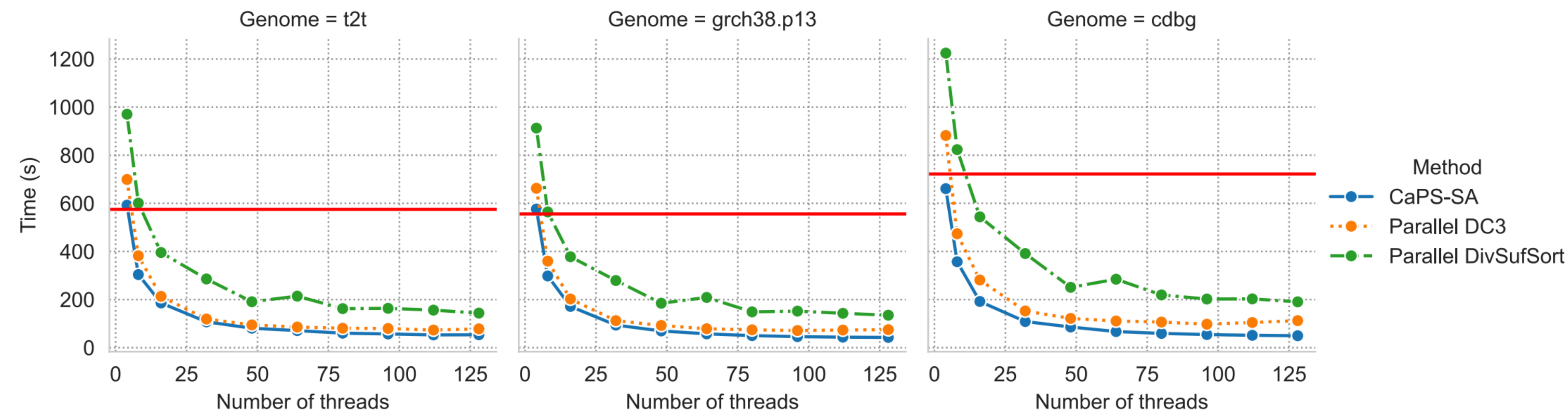


Fig. 3 Runtimes of CAPS-SA, PARALLEL-DIVSUF SORT, and PARALLEL-DC3 as thread count increases from 4 to 128 for GRCh38, T2T, and CdbG. `divsufsort` runtime is in red

Table 3 Cache-miss rates (in %) for compared methods on GRCh38 and T2T datasets with respect to number of threads

Method	Human (GRCh38)			Human (T2T)		
	1 thr	32 thr	64 thr	1 thr	32 thr	64 thr
CAPS-SA	24	32	37	43	53	60
PARALLEL-DIVSUF SORT	59	60	61	60	61	62
PARALLEL-DC3	72	68	68	72	68	68
divsufsort	51	-	-	52	-	-

Reported numbers are the minimum of 3 runs to obviate operating system jitter. Lower is better, and the best result is highlighted

Can easily compute a “partial” suffix array

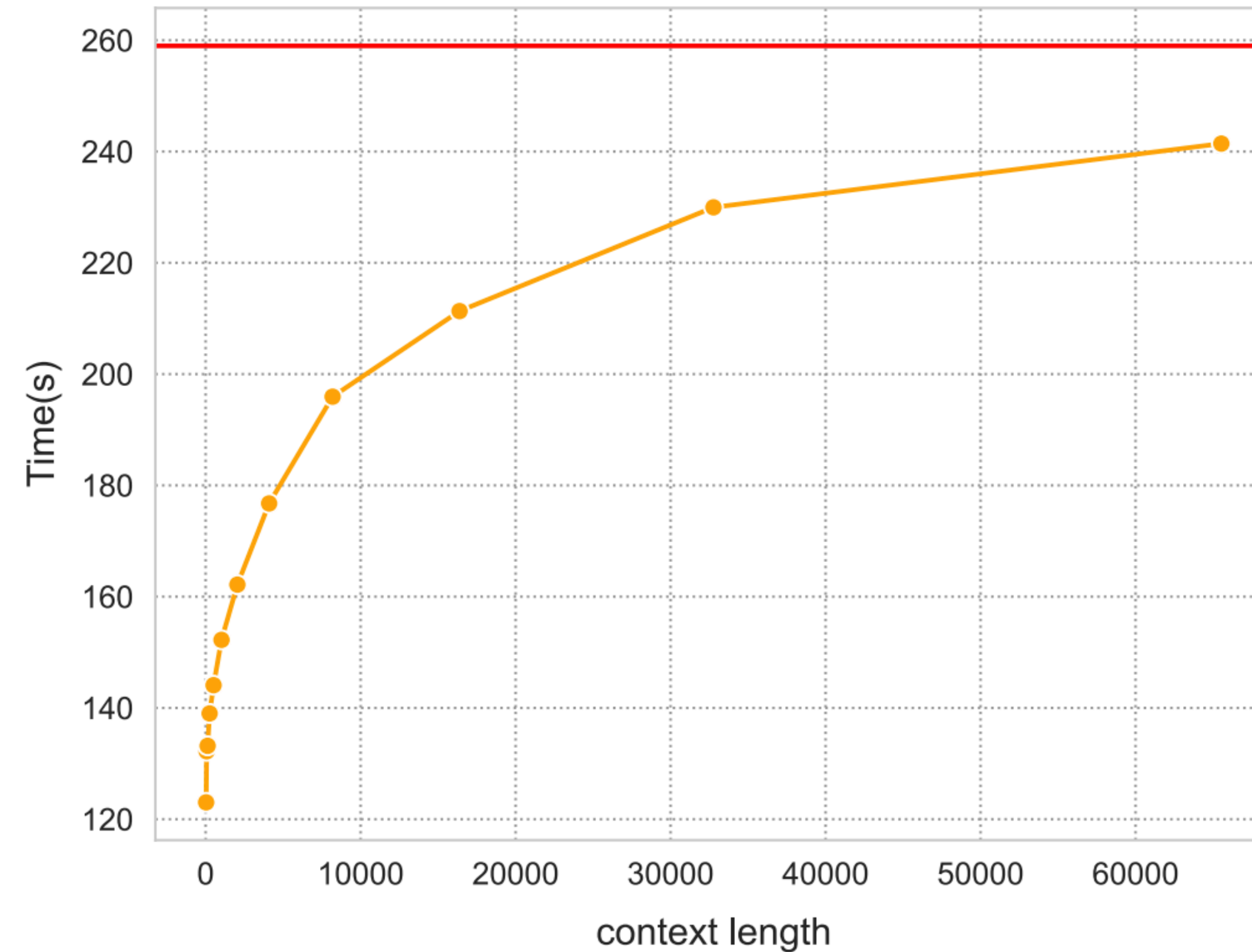


Fig. 4 Runtimes of CAPS-SA as context size increases on the bacteria (1K) dataset. Small context sizes greatly diminish the running time of the algorithm, but runtime is less sensitive for larger context sizes. The full-context runtime is shown as the red horizontal line